

# HydroServer: A software stack supporting collection, communication, storage, management, and sharing of data from in situ environmental sensors

Jeffery S. Horsburgh<sup>a,\*</sup>, Kenneth Lippold<sup>b</sup>, Daniel L. Slaugh<sup>b</sup>, Maurier Ramirez<sup>b</sup>

<sup>a</sup> Department of Civil and Environmental Engineering and Utah Water Research Laboratory, Utah State University, 8200 Old Main Hill, Logan, UT, 84322-8200, USA

<sup>b</sup> Utah Water Research Laboratory, Utah State University, Logan, UT, USA

## ARTICLE INFO

### Keywords:

HydroServer  
SensorThings  
Data model  
Environmental sensors  
Water resources  
Data management

## ABSTRACT

Scientists and practitioners who collect data using in situ environmental sensors need effective software for managing and sharing the data they produce. The HydroServer software stack was designed as a standards-based system that enables collection, storage, management, and sharing of environmental sensor data. Originally part of the Consortium of Universities for the Advancement of Hydrologic Science, Inc. (CUAHSI) Hydrologic Information System (HIS), the HydroServer software has been modernized to use the Open Geospatial Consortium's (OGC) SensorThings application programming interface (API) and data model standard to better enable flexible and standardized ingestion of data from multiple sensors and sources, along with query and retrieval of hosted data for a variety of different users and use cases. The HydroServer software is open source with deployment instructions targeted for cloud deployments, which makes it accessible and useable by both researchers and practitioners.

## Software availability

**Name of Software:** HydroServer.

**Developers:** Kenneth Lippold, Daniel Slaugh, Maurier Ramirez, and Jeffery S. Horsburgh.

**Contact:** [jeff.horsburgh@usu.edu](mailto:jeff.horsburgh@usu.edu).

**Year First Available:** 2024.

**Programming Languages:** The HydroServer software was primarily developed using TypeScript with the Vue JavaScript framework for frontend development and Python for application programming interface (API) and backend development.

**Hardware Required:** The HydroServer software stack was designed for deployment within a commercial cloud environment but can be deployed on a local machine. It requires a web server to run the web user interface and associated web services along with a database server to run a relational database management system (RDBMS). The web server and RDBMS instance can be hosted using managed cloud services or on any physical or virtual machine.

**Software Required:** HydroServer's data storage component requires the freely available PostgreSQL RDBMS (<https://www.postgresql.org/>).

For very large database implementations, the Timescale extension for PostgreSQL can be used, although it is not required. HydroServer's required database can be deployed within any physical or virtual machine hosting PostgreSQL or via a managed instance of PostgreSQL/ Timescale via commercial cloud providers such as Timescale Cloud (<https://www.timescale.com/>), Amazon Relational Database Service (<https://aws.amazon.com/rds/>), or Google Cloud SQL (<https://cloud.google.com/sql>). HydroServer requires Vue 3.4, Python Version 3.11 or later, Django Version 5.2 or later, and Django Ninja REST Framework Version 1.3 (<https://github.com/vitalik/django-ninja>).

**Software Availability:** HydroServer is available in GitHub at <https://github.com/hydroserver2>, with documentation, deployment/ installation instructions, and issue tracker for HydroServer available at <https://github.com/hydroserver2/hydroserver>. HydroServer software components have been split into multiple GitHub repositories for ease of management.

**Cost:** HydroServer is open-source, free of charge, and released using the Berkeley Software Distribution (BSD) 3-Clause open source license.

\* Corresponding author.

E-mail addresses: [jeff.horsburgh@usu.edu](mailto:jeff.horsburgh@usu.edu) (J.S. Horsburgh), [kenneth.lippold@usu.edu](mailto:kenneth.lippold@usu.edu) (K. Lippold), [daniel.slaugh@usu.edu](mailto:daniel.slaugh@usu.edu) (D.L. Slaugh), [maurier.ramirez@usu.edu](mailto:maurier.ramirez@usu.edu) (M. Ramirez).

<https://doi.org/10.1016/j.envsoft.2025.106637>

Received 4 May 2025; Received in revised form 1 July 2025; Accepted 23 July 2025

Available online 24 July 2025

1364-8152/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

## 1. Introduction

Environmental sensing technology has advanced rapidly over the past several years. The number of active monitoring sites and associated continuous datasets has increased as the value of continuous datasets has been recognized and as available sensing technology has improved and, in some cases, become less expensive. While the cost of research grade scientific sensors and dataloggers has generally not decreased, new technologies that enable much lower-cost sensing, data collection, and telemetry have emerged with the Internet of Things (IoT) (e.g., Wong and Kerkez, 2016; Trevathan and Schmidtke, 2022; Filhol et al., 2023) and low-cost, microcontroller-based dataloggers and sensing stations (e.g., Sadler et al., 2016; Campo et al., 2023). Effectively operating continuous monitoring sites of any level of complexity at scale requires not only the physical hardware required to measure and record observations (i.e., the sensors, dataloggers, and telemetry components installed in the field), but also supporting software that enables collection, communication, storage, management, and eventual sharing of the data. The integrated hardware and software systems used to support collection, storage, management, and sharing of water observations data and metadata are often referred to as Hydrologic Information Systems (HIS) (Spackman Jones and Horsburgh, 2025).

Given the prevalence of environmental sensing, ranging from networks of meteorological stations to streamflow gages, groundwater wells, continuous water quality stations, soil sensing, flow measurement at agricultural diversions, etc., HIS functionality is necessary for both researchers and practitioners whose day-to-day work includes continuous measurement of environmental variables. Practitioners may focus on a subset of these measurements - e.g., a state agency tasked with assessing and maintaining water quality may operate a network of continuous, in situ water quality measurement stations. Similarly, a state agency tasked with monitoring water allocation and use may operate a network of continuous diversion measurement stations and streamflow gages to track the quantity of available and diverted water. Within a research context, applications may range from smaller, focused studies with a few measurement stations and sensors (e.g., Scarbrough et al., 2023) to much larger, interdisciplinary environmental observatories that combine all of the measurement types mentioned above (e.g., Jones et al., 2017; O'Geen et al., 2018). Regardless of the application, cyberinfrastructure is needed to support the necessary data collection and management pipelines required by these types of organizations.

While software supporting environmental sensing systems has advanced, it has happened in ways that do not necessarily meet the needs of many environmental sensing use cases. Heterogeneity in sensing hardware and software that arises from inconsistency in data collection, storage, and communication protocols used by different sensor/datalogger manufacturers makes it difficult to assemble a reliable network of in situ sensors from different manufacturers for a research watershed or site. Inconsistencies between communication protocols and software from different sensor/datalogger manufacturers may require researchers to work across multiple systems to meet their needs.

More generalized time series data management systems can help address these issues, but there remain few complete, generalized, off-the-shelf software solutions that enable collecting, communicating, storing, managing, and sharing time series of observational data from environmental sensors. Commercial options range from excellent (but expensive and proprietary) software systems like Aquatic Informatics Aquarius Time Series (<https://aquaticinformatics.com/products/aquarius-environmental-water-data-management/>) or Kisters WISKI (<https://www.kisters.net/wiski/>) to more specialized software solutions that remain specific to a particular sensor or datalogger manufacturer (e.g., Campbell Scientific's Loggernet software (<https://www.campbellsci.com/loggernet>) and associated utilities that enable data collection, telemetry, storage, and sharing of data from Campbell Scientific dataloggers). Commercial software can provide

solutions for some, but cost and/or linkage to a particular set of hardware may place them out of reach for others.

Advanced, open source software options are few. In one very recent example, Bumberger et al. (2025) describe a "digital ecosystem for FAIR time series data management in environmental system science" made up of three major components: a sensor management system for registering sensors and managing sensor metadata, a data infrastructure for storing and managing time series data, and a system for automated quality control for managing data quality (Schmidt et al., 2023). Their modular software demonstrates a viable architecture that may work well for some users and applications, but registration of persistent identifiers and user authentication and identity management provided by European-specific services along with the specific technologies used by their system that promote interoperability among European partners, may not work more broadly. Additional options are needed that meet functional use case requirements but allow different groups of users to choose an architecture and software implementation that works well for their needs.

Additional limitations of existing software include lack of support for IoT devices, IoT protocols, and open standards for interfaces and data encodings. This can constrain some use cases for which lower cost, resource constrained IoT devices may be well suited, and may also constrain use cases that require interoperability across software systems. Generic software platforms supporting data collection using IoT devices are available (Diène et al., 2020), but are usually IoT specific and application agnostic because IoT standards must support a broad range of monitoring applications (e.g., environmental monitoring, healthcare systems, building automation systems, etc.). This means that some customization is usually required for their use in any particular domain, which may be difficult for many environmental monitoring organizations that lack dedicated software developers. IoT software systems also often assume that data are short lived with little support for the type of long term data management required for environmental monitoring. With regard to interface and encoding standards, while software options may differ in technical implementations, adopting similar interfaces and data encodings such as the Open Geospatial Consortium's (OGC) SensorThings application programming interface (API) and data model standard (Liang et al., 2021 - hereafter referred to as "STA") as demonstrated by Bumberger et al. (2025) can make software interoperable despite technical differences.

In the context of HIS, lack of standardization and related interoperability are issues for both data ingestion and for querying and retrieving data. It is common for environmental sensor networks to be made up of different kinds of monitoring stations (e.g., weather stations, streamflow gages, water quality monitoring stations, etc.) that may require sensors, dataloggers, and communication peripherals from multiple different manufacturers. Thus, for many data collectors, generalized methods are needed for ingestion of data into management software so that data can be consistently loaded, stored, and managed. Providing consistent interfaces for querying, accessing, and retrieving data is also important on the data consumer end to ensure that regardless of how data are collected and loaded they are made available using standardized protocols, interfaces, and encodings that support automated retrieval and simple interoperability with data visualization and analysis software. Spackman Jones and Horsburgh (2025) summarize these and several other challenges that drove the software modernization efforts described in this paper.

In this paper we describe a modernized version of the HydroServer software, which is a generalized HIS that was designed to support collection, communication, storage, management, and sharing of data from in situ environmental sensors. Originally part of the Consortium of Universities for the Advancement of Hydrologic Science, Inc. (CUAHSI) HIS (Horsburgh et al., 2010), HydroServer has been redesigned and modernized using the OGC's STA, which specifies standard web service endpoints for both ingesting and retrieving data along with standardized data encodings. HydroServer was designed to provide a generalized solution for bridging across various sensors, dataloggers, and

communication systems that organizations use in their environmental monitoring data collection workflows. This paper builds on a prior publication that described HydroServer's implementation of STA (Horsburgh et al., 2025) to provide a description of the full HydroServer software stack and capabilities. Thus, the specific contributions of this paper include: 1) an open-source, deployable HIS software stack supporting collection, management, and sharing of data from in situ environmental sensors; 2) integration and implementation of standards-based web service interfaces and data encodings that promote interoperability for both data ingestion and data query and retrieval; and 3) design and development of web user interfaces and a Python client package that allow creation and management of monitoring station, observed variable, and datastream metadata along with their associated sensor data within a multi-user HIS.

## 2. Background

In its very early versions (Horsburgh et al., 2010), HydroServer was developed as part of the CUAHSI HIS (Hooper et al., 2004; Maidment, 2008; Horsburgh et al., 2009; Tarboton et al., 2009) to enable producers of hydrologic data (defined by this early project as time series of sensor data collected at fixed monitoring sites for a given study area or watershed) to establish a data management server for a study area or region, populate any number of databases conforming to the Observations Data Model (ODM) (Horsburgh et al., 2008) hosted on that server with observational data from their continuous monitoring stations, and then connect those ODM databases to WaterOneFlow web services also hosted on that server that published the observational data contained within them on the Internet for easy access. WaterOneFlow was an early attempt at a standardized web service interface for data querying and retrieval that used a machine-readable eXtensible Markup Language (XML) schema called WaterML for encoding data for transfer over the Internet (Zaslavsky et al., 2007).

As a way of grouping data, users decided which ODM databases and WaterOneFlow web services to establish (e.g., one for each research project or one for each experimental watershed). Each WaterOneFlow web service was registered with a central metadata catalog hosted by CUAHSI, which harvested metadata from each registered service and provided data discovery capabilities across all registered services (Ames et al., 2012). The initial HydroServer implementation was built using Microsoft.Net, Microsoft SQL Server, and other proprietary software programs, but a lighter weight, open source version of HydroServer was also developed by an unrelated research group (Connor et al., 2013). HydroServer and the CUAHSI HIS comprised one of the first services-oriented architectures for sharing hydrologic observations.

While elements of the CUAHSI HIS are still operated by CUAHSI, they moved away from a distributed server/service model and instead began hosting ODM databases and WaterOneFlow web services within CUAHSI's commercial cloud account with similar catalog/discovery capabilities provided by their "HydroClient" web application (<http://data.cuahsi.org>). This is a valuable data sharing service that CUAHSI has provided for the hydrologic science community in the United States. However, when the CUAHSI HIS transitioned from using distributed HydroServer instances to a more centralized system operated by CUAHSI for a broader community of users, important functionality was lost. For example, WaterOneFlow provides a public data query API, but does not allow data upload. CUAHSI did not advance or maintain HydroServer's Streaming Data Loader software, which allowed for near real-time ingestion of sensor data into an ODM database, instead developing a file-based data uploader that enables users to manually add data to their CUAHSI-hosted ODM databases. As a result, dynamic uploading of streaming sensor data was disabled, making the current HIS operated by CUAHSI useful for data sharing, but not for active, operational, day-to-day data management.

Advances in available standards for web service interfaces and data encodings have also happened since HydroServer was originally

conceptualized. The CUAHSI WaterML data encoding (Zaslavsky et al., 2007) was advanced as an OGC Implementation standard called WaterML 2.0 that now has four parts: 1) a standard information model and XML data encoding for representation of water observations data (Taylor, 2014); 2) a standard information model and XML data encoding for hydrologic ratings, gauging observations, and cross sectional surveys (Taylor, 2015); 3) a conceptual information model for representing hydrologic features and their geometric representation (Blodgett and Dornblut, 2018), and 4) an information model and XML encoding for groundwater data (Brodaric, 2017). OGC also developed and advanced a suite of web service interface standards and data encodings for "Sensor Web Enablement", including the OGC Observations & Measurements Schema (O&M) for encoding observations and measurements from a sensor (Cox, 2013); OGC Sensor Observation Service (SOS) for requesting, filtering, and retrieving observations and sensor system information (Bröring et al., 2012); the OGC Sensor Planning Service for requesting user-driven acquisitions and observations (Simonis and Echterhoff, 2011); and Sensor Model Language (SensorML), which is an information model and XML encoding schema for describing sensor systems and processes (Botts et al., 2020).

More recently, to address advances in IoT sensing and to introduce modern representational state transfer (REST) principles and efficient JavaScript object notation (JSON) data encodings, OGC released a two-part SensorThings API and data model that enables creating, managing, and accessing observations from dataloggers, IoT devices, and other software applications along with a standard way for tasking IoT devices (Liang et al., 2021; Liang and Khalafbeigi, 2019). Thus, the landscape of available web service interfaces and data encoding standards has advanced significantly since HydroServer was originally developed, which required updating the software stack for it to be useful in meeting more recent sensing use cases and requirements using available standards.

The need for software tools that enable operational management of streaming sensor data has grown considerably over the past decade. The need for modernized tools that better support newer data collection technologies, including IoT devices, has also grown. As one example, the authors manage operational data from the Logan River Observatory (LRO) (<https://uwr.usu.edu/lro/>, Neilson et al., 2021), which consists of nearly 30 continuous monitoring sites generating new data every 15 min from a broad variety of aquatic and terrestrial sensors. With more than 10 years of data at some sites, the LRO observations database now contains more than 253 million individual observations, with approximately 43,000 new raw observations added daily. This scale of data collection is not uncommon in research watersheds, experimental sites, and within operational monitoring networks worldwide. Thus, we saw the need to modernize the HydroServer software stack to meet these needs.

This paper describes development of a modernized and advanced version of the HydroServer software stack using the OGC SensorThings API and data model as core components. In an earlier paper (Horsburgh et al., 2025), we described HydroServer's adaptation of three HydroServer components: 1) the underlying data model for storing hydrologic observations data, 2) a modern, standardized web service interface for data ingest and data query, and 3) an interoperable data encoding for exchanging data over the Internet. That paper laid out requirements for these three components, including efficient and performant data storage, sensor data encoding, sensor data ingest, data query, scalability, and multiuser support. Here we expand beyond those requirements to describe HydroServer's full architecture and the rest of the HydroServer software stack that meets the following functional use cases and requirements.

- **A web user interface:** A web-based, graphical user interface is required to enable users to interact with the system, create metadata about monitoring sites, observed properties, sensors, etc., and access, visualize, and export data.

- **Tools for data ingestion from a variety of sources:** Software tools must support ingesting data from a variety of sources, including Internet connected dataloggers, loading data from files, and scripts designed for scraping data from external sources.
- **Functionality for managing the quality of collected sensor data:** Tools must be available for performing review and editing of raw data entered into the system to produce new versions of the data that have been quality controlled.
- **Client tools enabling data query, visualization, and analysis:** Client tools must be available to facilitate data retrieval via queries as well as loading data into analytical environments for visualization and analysis.
- **Flexible deployment to local server machines or the commercial cloud:** The entire system must be deployable to local physical or virtual machines for local control or to commercial cloud systems for scalability.

### 3. Methods

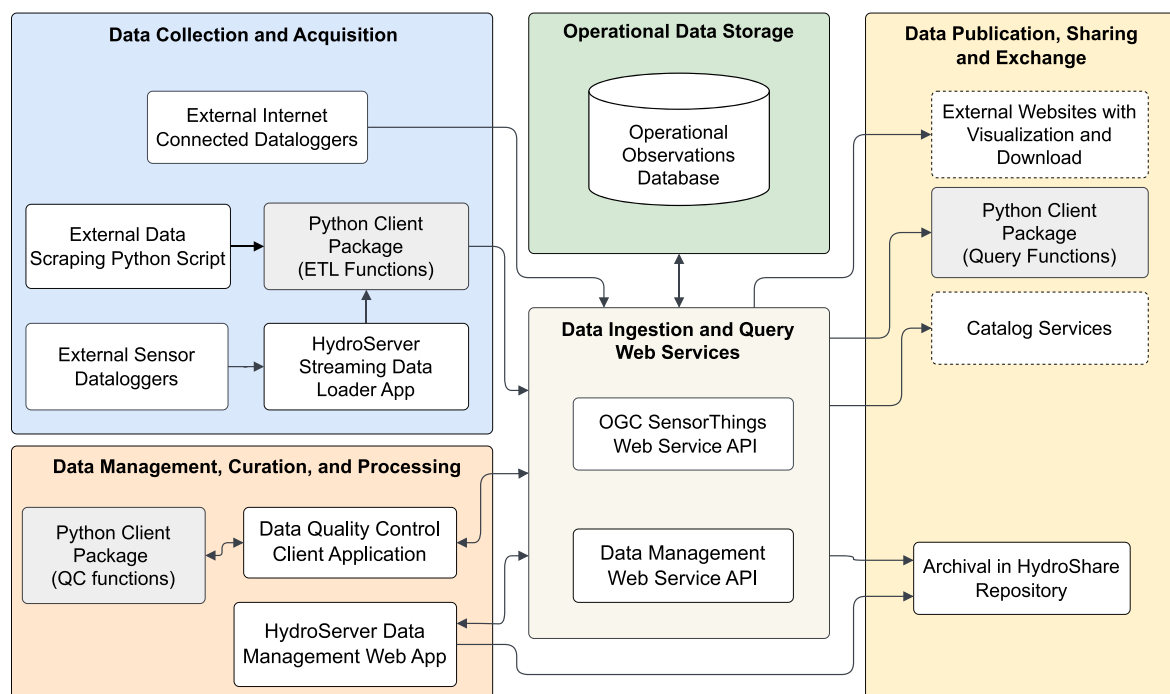
#### 3.1. HydroServer design and overall software architecture

Our goal in developing HydroServer was to create a generalized HIS software stack that meets the day-to-day data collection, storage, management, and sharing needs for researchers and practitioners who collect continuous data from environmental sensors, including our own team. We based our design for HydroServer's architecture on the functional use cases and requirements listed in Section 2, earlier versions of the HydroServer software, and the generalized HIS architecture extracted from a recent review of relevant HIS literature and operational HIS systems presented by [Spackman Jones and Horsburgh \(2025\)](#). These three sources served to define the requirements, assumptions, and criteria for HydroServer's architectural design. This generalized architecture consists of four primary components: 1) data collection and acquisition where data are observed and then transferred to an operational storage system; 2) operational storage where data are stored along with descriptive metadata; 3) data management, curation, and

processing where data are processed, quality controlled, versioned, etc.; and 4) publication, sharing, and exchange where data are accessed by end users. HydroServer's overall software architecture ([Fig. 1](#)) follows this general HIS architecture pattern, but explicitly groups the web service interfaces for data ingestion and query in a central component (bottom center box in [Fig. 1](#)). In the sections that follow, we describe high-level design considerations for each of the major components of HydroServer's architecture shown in [Fig. 1](#). In the Results section, we describe HydroServer's specific software implementation, including technologies used in implementation and the functionality of each component.

#### 3.2. Design of data collection and acquisition

Our objective with the data collection and acquisition component of HydroServer (top left box in [Fig. 1](#)) was to enable data entry via several mechanisms that would cover a large number of use cases in hydrologic and environmental monitoring. We did not want to replace existing proprietary software used for interfacing with sensors and dataloggers in the field (e.g., systems like Campbell Scientific's Loggernet software), which we assume people will continue to use, but rather focused on a design that includes a web service API for data ingestion (lower center box in [Fig. 1](#)) along with software that facilitates loading data from common data formats produced by proprietary software systems (e.g., comma separated values (CSV) files). Thus, the design for this component shows three main pathways for loading data into HydroServer: 1) Internet connected dataloggers can post data directly through HydroServer's web service API; 2) any script or software program can load data through HydroServer's web service API either directly or through HydroServer's Python client package; and 3) CSV files produced by any software system that interfaces with field deployed sensors/dataloggers can be loaded using a "Streaming Data Loader" software app. The flexibility provided by a data ingestion API and a Python client package with extract, transform, and load (ETL) functions that facilitate its use means that data can be loaded from any source, and jobs for loading data can be automated using whatever tool is best suited for the user or job – e.g.,



**Fig. 1.** Modernized HydroServer architecture showing major HIS components and how they are connected. Larger colored boxes represent major architectural components described in subsequent sections. Smaller boxes represent software components built to implement the architecture. Arrows between boxes represent metadata and data flow.



scheduling a Python script as a regularly executed Windows task or using cloud-based orchestration systems like Google Cloud Composer (<https://cloud.google.com/composer>) to automate data loading.

### 3.3. Design of operational data storage and web services

Within the review of HIS systems and generalized architecture provided by Spackman Jones and Horsburgh (2025), operational data storage (top center box in Fig. 1) is described as providing a location, structure, and technology for storing and describing data. Connected to the operational data store are services that provide the ability to upload and retrieve data using standardized web service interfaces and data encodings (bottom center box in Fig. 1). To meet these needs, HydroServer was designed to use an extended version of the OGC STA data model (Liang et al., 2021) for its operational data store, to which we added metadata necessary for unambiguous description of environmental monitoring data from the Observations Data Model (ODM – Horsburgh et al., 2008; Horsburgh et al., 2016). We also added additional entities and attributes to the data model to enable HydroServer to handle data from multiple users and to provide security and access control. Thus, the data model serves as both the structure for HydroServer's database (labeled as "Operational Observations Database" in Fig. 1) and as the basis for encoding data for input to or output from HydroServer's web service interfaces (arrows pointing into and out of the bottom center box in Fig. 1). HydroServer's web service interfaces (bottom center box in Fig. 1) include an implementation of the OGC STA Part 1: Sensing Version 1.1 API (Liang et al., 2021), to which we added a data management web service API to assist with user management and creation and management of entities we added to the STA from ODM. Horsburgh et al. (2025) provide a detailed description of the design for HydroServer's operational data storage and web services, including the mapping from ODM entities and attributes to entities and attributes from STA and the considerations for design of HydroServer's APIs.

### 3.4. Design of data management, curation, and processing

In its current implementation, HydroServer meets two major needs via the data management, curation, and processing component (bottom left box in Fig. 1). First, we designed a data management web application that provides a graphical user interface enabling users to access HydroServer's major functionality. This includes creating user accounts; creating workspaces; creating and managing metadata for monitoring sites, observed properties, sensors, datastreams, etc.; browsing, inspecting, and visualizing available data; managing access control for monitoring sites and associated datastreams; and setting up data loading workflows. All of this functionality can also be scripted through HydroServer's web service APIs for automation, but the data management web application provides a straightforward way for users to access needed functionality through a web browser without writing code.

Second, HydroServer provides access to functionality for performing quality control (QC) on raw data stored within the system. We designed QC functionality for scriptability via data QC functions coded within HydroServer's Python client package. These functions are useable in stand-alone Python scripts or via a QC software application (labeled "Data Quality Control Client Application in Fig. 1) that provides a graphical user interface for using those functions and recording/saving results. The Python client package is described in more detail below, but the QC software application is currently under development and is not described in detail in this paper.

### 3.5. Design of data publication, sharing, and exchange

While shown as a separate component in Fig. 1 (right box), the data publication, sharing, and exchange component of HydroServer relies on the data query services provided by the OGC STA for its functionality

(Fig. 1, Data Ingestion and Query Web Services). STA provides a standardized web service interface and JavaScript object notation (JSON) encoding for data and metadata query and retrieval that can be used by any client program for retrieving available data from a HydroServer instance. We designed HydroServer's Python client package (shown as a grey component in multiple locations in Fig. 1) to provide convenience functions for querying and retrieving data as well as for loading data into performant data structures for analysis within a Python environment. While HydroServer's design includes a Python client package, HydroServer's STA implementation is accessible from any programming language capable of using modern REST patterns for retrieving data.

The components shown with dotted lines in the right box of Fig. 1 (external websites with visualization and download and catalog services) are included in our architecture design but are not formally part of the HydroServer software stack. We anticipate that others may wish to design and build web applications that provide advanced data access (e.g., visualization dashboards, displays of real-time data, etc.) and/or data catalog and discovery services on top of the HydroServer infrastructure (as CUAHSI did for the original CUAHSI HIS), which is made possible through HydroServer's open, web services-oriented architecture.

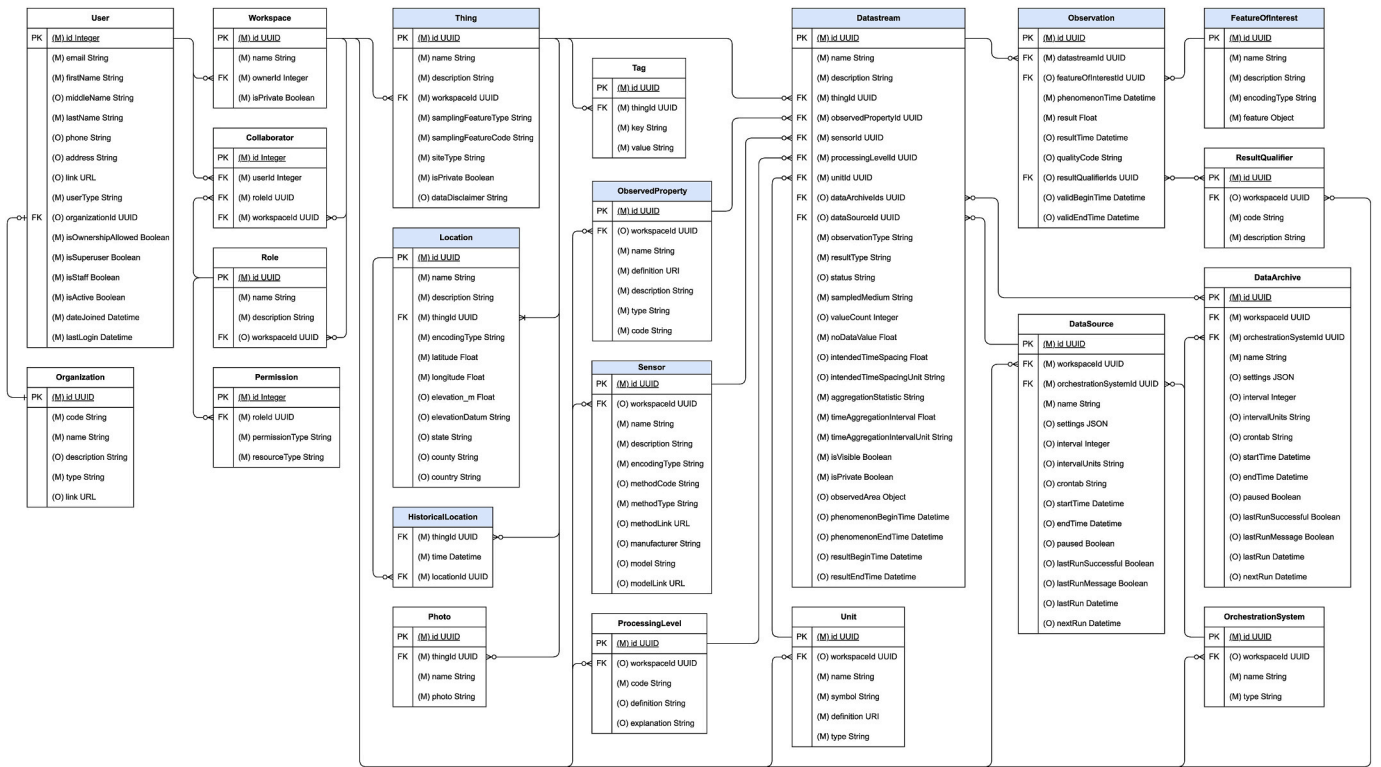
Finally, we designed HydroServer to use HydroShare (Horsburgh et al., 2015a; Tarboton et al., 2024) as a target for long term data archival (bottom component in the right box of Fig. 1). HydroShare is a data and model sharing repository that uses a file-based data model for archiving and sharing water science related data and research products. There are two use cases met by this functionality: 1) archiving time series data to HydroShare in a simple to use data format (i.e., CSV files) provides an easy way to give access to operational data for users who simply want to download and begin working with data but who may not have the expertise or desire to write code to retrieve the data from HydroServer's APIs, and 2) it provides a mechanism for archiving time series data at the end of operational data collection – e.g., when a project ends, active data collection is no longer ongoing, and the data become static. When this happens, data within a HydroServer instance can be archived to HydroShare and then the HydroServer instance can be shut down.

## 4. Results

The following sections provide implementation details for the HydroServer software components shown in Fig. 1. For demonstration purposes, we have deployed an instance of HydroServer for the LRO at <https://lro.hydroserver.org> which contains time series data for stations in the Logan River watershed of northern Utah aimed at collecting comprehensive hydrologic data to inform water management decision making and support hydrologic research. LRO data is openly and freely available. We also provide a public testing instance of HydroServer at <https://playground.hydroserver.org> that allows users to create a user account and test HydroServer's functionality within a playground server environment.

### 4.1. Data model, operational database, and web service APIs

In Horsburgh et al. (2025) we described the central components in Fig. 1, including the design, metadata considerations, physical implementation, and benchmark testing of HydroServer's data model, operational observations database, and data ingestion and query web service interfaces as an adaptation of OGC's STA implementation standard (Liang et al., 2021). Thus, in the following sub-sections we provide only a high-level overview of these components and a visualization of HydroServer's data model (Fig. 2) for completeness, to establish terminology, to highlight changes that have been made since the earlier paper was published, and for reference in subsequent sections.



**Fig. 2.** HydroServer's data model shown as an entity relationship diagram with common crow's foot notation and adapted from OGC's STA implementation standard for implementation within a RDBMS. Entities shown with blue title bars are original to the STA standard, whereas entities with white title bars were added to meet specific environmental monitoring use cases supported by HydroServer. For additional detail, see [Horsburgh et al. \(2025\)](#). (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

#### 4.1.1. HydroServer data model

HydroServer adopted important entities from the STA data model (Fig. 2), including “Thing”, which in the context of HydroServer is a monitoring site at which observations are made; “Location”, which is the physical location of a Thing; “Sensor”, which is the physical sensor or procedure used to make the observations; “ObservedProperty”, which is the phenomenon that was observed by a Sensor; “Datastream”, which is a time series collection of Observations for the same ObservedProperty produced by the same Sensor on, at, or associated with a Thing; and “Observation”, which is the act of measuring or otherwise determining the value of a property including its timestamp and observed value. To these core STA entities we added “Photo”, which is used to store information about photographs of monitoring sites; “Tag”, which is used to store user defined key-value pair metadata for monitoring sites; “ProcessingLevel”, which is used to store information about the degree of quality control or processing a datastream has been subject to; “Unit”, which is used to store information about units of measure; and “ResultQualifier”, which is used to store data qualifying comments.

We further extended the STA data model to support major functionalities that go beyond what is provided by STA but that are needed to operate HydroServer as a multi-user software environment. Because STA does not include user information or the concept of ownership within the data model or API, we extended the STA data model to enable ownership and access control. We originally implemented access control at the individual monitoring station (Thing) level but found that this required a lot of work for users to maintain as the number of monitoring stations grew. As an alternative, we implemented role-based access control over “workspaces”. When authorized by a HydroServer administrator, users can create workspaces within which they can create and share monitoring stations, datastreams, and associated metadata. Users can then add other users to a workspace with roles of owner, editor, and viewer. Entities on the left side of Fig. 2, including “User”,

“Organization”, “Workspace”, “Collaborator”, “Role”, and “Permission” enable HydroServer's access control.

Finally, to facilitate the orchestration of automated data loading using HydroServer's Streaming Data Loader software application and potentially other data loading software along with automated archival of data from HydroServer to HydroShare, we added the “DataSource”, “DataArchive”, and “OrchestrationSystem” entities to the data model. DataSource and OrchestrationSystem store information about sources of data from which data are loaded into a HydroServer instance. These tables store information required to map external sources of data (e.g., files downloaded from dataloggers at monitoring stations) to datastreams that have been set up in HydroServer's database. These mappings are made available to data loading software via HydroServer's data management API (Section 4.1.3). The DataArchive table stores information about destination HydroShare resources where data exported from HydroServer will be archived along with the jobs that run data archival. A full data dictionary for the HydroServer data model is available in HydroServer's documentation (<https://hydroserver2.github.io/hydroserver/>).

#### 4.1.2. HydroServer operational database

HydroServer uses an implementation of the data model design in Fig. 2 for storing data and metadata within the PostgreSQL (<https://www.postgresql.org/>) relational database management system (RDBMS). We chose PostgreSQL because it integrates seamlessly with the web development technologies we chose and is widely used, open-sourced, free to use, reliable, performant, and feature rich. For very large HydroServer instances, the Timescale extension for PostgreSQL may provide some performance benefits by adding time series database features but is not required. HydroServer uses globally unique identifiers (GUIDs) to uniquely identify entities in its database to ensure that they are unique across the computers and networks that may contribute or

consume data from a HydroServer instance. GUIDs guarantee uniqueness across different systems that might interface with a HydroServer and meet the requirements for unique identifiers within HydroServer's STA implementation. HydroServer's operational database may be hosted on a local database server or can be implemented on a cloud-based, virtual database server or via a managed instance of PostgreSQL available from a commercial cloud provider. Each of these options has advantages and disadvantages related to cost, scalability, and performance that are beyond the scope of this paper to describe, but that may impact choices related to deployment environment (described in more detail in Section 4.6).

#### 4.1.3. HydroServer web service API implementation

HydroServer's OGC STA implementation is a software package developed using the Python programming language, with requests and responses that are consistent with the OGC STA specification. Horsburgh et al. (2025) provide justifications for the specific technology choices we made in implementing HydroServer's APIs. HydroServer uses STA's extensibility mechanisms to store and encode extended metadata added to STA's data model from the ODM time series data model (Horsburgh et al., 2016). Consistent with the OGC specification, HydroServer's STA implementation follows the Open Data Protocol's (OData) specification and URL conventions for requesting entities (Pizzo et al., 2020), and resource paths, query options, and JSON encodings of responses follow OData 4.0 (Handl et al., 2016).

Because we added metadata entities to the SensorThings data model along with user management, access control, and orchestration of data loading, which are not handled by the OGC STA specification, we developed a data management web service API that is complementary to STA (see Appendix A of Horsburgh et al. (2025) for a description of endpoints available via HydroServer's data management API). Together, HydroServer's STA and data management web service API provide the ability to comprehensively create, read, update, and delete (CRUD) metadata and observational data within a HydroServer instance and are used by HydroServer's user-oriented software applications to perform these operations.

### 4.2. Python client package

We developed a Python client package called *hydroserverpy* to facilitate and simplify coding interactions with HydroServer's web service APIs. The *hydroserverpy* package abstracts low-level details of making HTTP requests (including which of HydroServer's APIs a request needs to be sent to), handles user authentication, and simplifies the code needed to automate many data loading, query, and management tasks. The *hydroserverpy* package also handles the conversion of JSON responses from the API into objects that are easy to work with in code. We chose Python for its relatively simple syntax, widespread use, and availability of libraries like Pandas for working with time series data. The *hydroserverpy* package currently consists of three sub-packages described in the following sections, with more detailed descriptions of the *hydroserverpy* classes provided in Appendix A. Detailed documentation of *hydroserverpy*, including extensive code examples, is available in the *hydroserverpy* GitHub repository (<https://github.com/hydroserver2/hydroserverpy>), and the package is available for installation via the Python Package Index (PyPI).

#### 4.2.1. Hydroserverpy api sub-package

The *api* sub-package provides functionality for creating a connection to a HydroServer instance and then exposes core data and metadata for the Things, Datastreams, Sensors, Units, ProcessingLevels, ObservedProperties, and ResultQualifiers entities for creation, editing, retrieval, or deletion using *list*, *get*, *create*, *save*, or *delete* methods associated with the object representation of an entity instance. An additional *refresh* method allows users to replace any changes that have been made to their local object representation of an entity instance with whatever is

currently in HydroServer. The *api* sub-package also provides functionality that is similar to the United States Geological Survey's (USGS) dataretrieval package (Hodson et al., 2023) for retrieving datastreams or datastream subsets and loading them into a Pandas DataFrame object, simplifying code required to retrieve, parse, and organize data for visualization and/or analysis.

#### 4.2.2. Hydroserverpy etl sub-package

The *etl* (extract, transform, and load) sub-package provides functionality for loading data into a HydroServer instance, including creating and managing information needed to extract data from a source, transform it into a format ready for loading into HydroServer, and then load the data into HydroServer's database through the STA. The *etl* sub-package is used by HydroServer's Streaming Data Loader software (described below) for loading data from CSV data files (data sources) and can be used to manage the list of Streaming Data Loader instances connected to a HydroServer, the list of data sources associated with each Streaming Data Loader Instance (CSV files from which data are to be loaded), and the list of datastreams associated with each data source (columns in the CSV file containing observations that correspond with a datastream in HydroServer).

While it was originally developed to manage information needed for the Streaming Data Loader, we added functionality to the *etl* sub-package to enable generic data loading – e.g., loading data from any source of time series data. We implemented a set of extractors capable of authenticating with and retrieving data from various endpoints using HTTP, SFTP, and other data transfer protocols. We then implemented a set of transformers that can be used to convert whatever format is received from an extractor into a pandas DataFrame object. Finally, we defined a loader that uses a generic *load\_observations* method on a datastream that takes as input a pandas DataFrame object containing series of timestamps and numeric values to be loaded into that datastream. This generic method can be used to load data from any source that can be parsed into a pandas DataFrame object.

Additionally, the classes within the *etl* sub-package are designed to be extensible, allowing users to implement their own custom extractors, transformers, and loaders. This flexibility enables the integration of non-standard or legacy data formats into HydroServer. By creating new classes or using the provided ones, users can tailor the data loading process to accommodate unique data sources and formats, ensuring compatibility with the HydroServer platform by enabling seamless data ingestion from a variety of sources.

#### 4.2.3. Hydroserverpy quality sub-package

The *quality* sub-package contains a set of functions that can be scripted to create a reproducible QC workflow for a datastream – i.e., a Python script for converting raw sensor data to quality-controlled data. Functions include filters to find and select data values within a datastream that may need processing, including value threshold selection (values greater than, less than, equal to, etc. a threshold), gaps (instances where the time between recorded data values exceeds a threshold), value change thresholds (instances where two consecutive data values differ by more than a threshold), and persistence (instances where the same data value is reported for a time period longer than a threshold). The *quality* sub-package also includes editing functions that operate on selected data values, including inserting values, filling gaps, deleting values, interpolation, adjusting values using a mathematical operation, shifting values in time, and linear drift correction. The *api* sub-package can be used to retrieve a datastream from HydroServer for processing, and then the *quality* sub-package can be used to perform necessary QC edits. Results can then be written back to the HydroServer database using the *api* package. While the functions in the *quality* sub-package can be used in any Python environment to create reproducible quality control scripts, we are currently developing a software app that implements these functions within a graphical user interface that combines data plotting, visual data point selection, and automated



recording of user edits as QC function calls in a recorded “history” that enables a QC workflow to be reproduced – i.e., a modernized and optimized version of a desktop app we developed for the original HydroServer that had several limitations related to performance and security (Horsburgh et al., 2015b).

#### 4.3. Data management web app

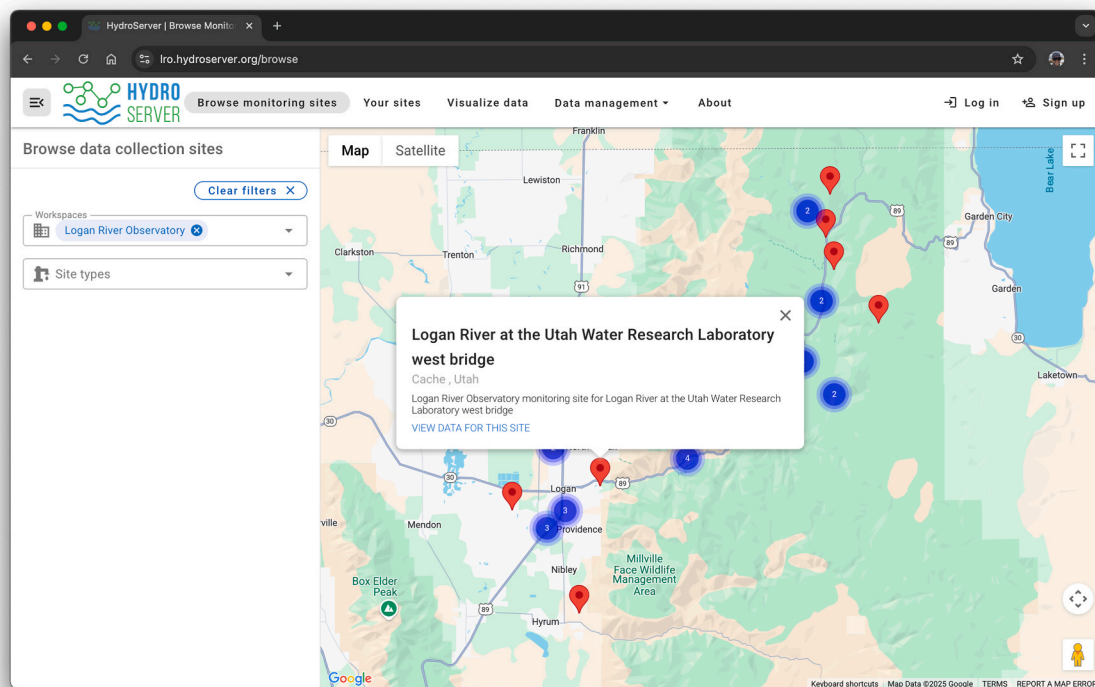
HydroServer’s data management web app provides a graphical user interface that allows users to create and manage monitoring sites and their associated datastreams and metadata. It combines common HIS data consumer tools described by Spackman Jones and Horsburgh (2025) – e.g., a map for displaying locations of monitoring sites, a time series plot viewer for visualizing observed variables at selected locations, a landing page for each monitoring station, and options for data download – with metadata creation and management functionality required by data managers. The data management app was developed using the Vue.js 3.4 framework (<https://vuejs.org/>) with TypeScript 5.0 (<https://www.typescriptlang.org/>), leveraging a web development stack that includes Vuetify 3.7 (<https://vuetifyjs.com/en/>) for UI components, Pinia 2.1 (<https://pinia.vuejs.org/>) for state management, Vitest 1.1 (<https://vitest.dev/>) for testing, and the Google Maps JavaScript API (<https://developers.google.com/maps/documentation/javascript/>) via the Google Maps JavaScript API Loader 1.16 (<https://github.com/googlemaps/js-api-loader>) for mapping functionalities. These specific technologies were chosen because they are currently widely used, they meet the specific functional use cases required by the data management app, they were familiar to our development team, and they are compatible with other projects and software development efforts on which we work. The following sections describe each of the major pages within the data management web app and their functionality.

##### 4.3.1. Browse monitoring sites page

The “Browse monitoring sites” page (Fig. 3) provides a map interface for viewing and orienting users geographically with respect to monitoring site locations. The map interface provides a way for users to see monitoring locations in relation to each other, to stream and water body features, and other geospatial features represented in the base map services. We implemented this page using the Google Maps JavaScript API with custom markers and pop-up bubbles when a user clicks on a monitoring site location on the map. We chose the Google maps API because it provides a straightforward way to provide an interactive web map with detailed base map services for referencing users geographically. Filters are provided to limit the monitoring sites that are shown in the map. Currently, users can filter by workspace or by site types, although additional filters could be implemented based on monitoring site metadata stored in HydroServer’s database. The “Browse monitoring sites” page defaults to show all sites contained within workspaces for which the user has at least a “viewer” role along with any sites that have been made publicly visible by other users. Clicking on a site location provides a bubble pop-up with a link to “View Data for this Site”, which takes the user to the landing page for the monitoring site where they can access site metadata and datastreams. Thus, users can view, manage, and download their own data as well as view and download public data shared by other users.

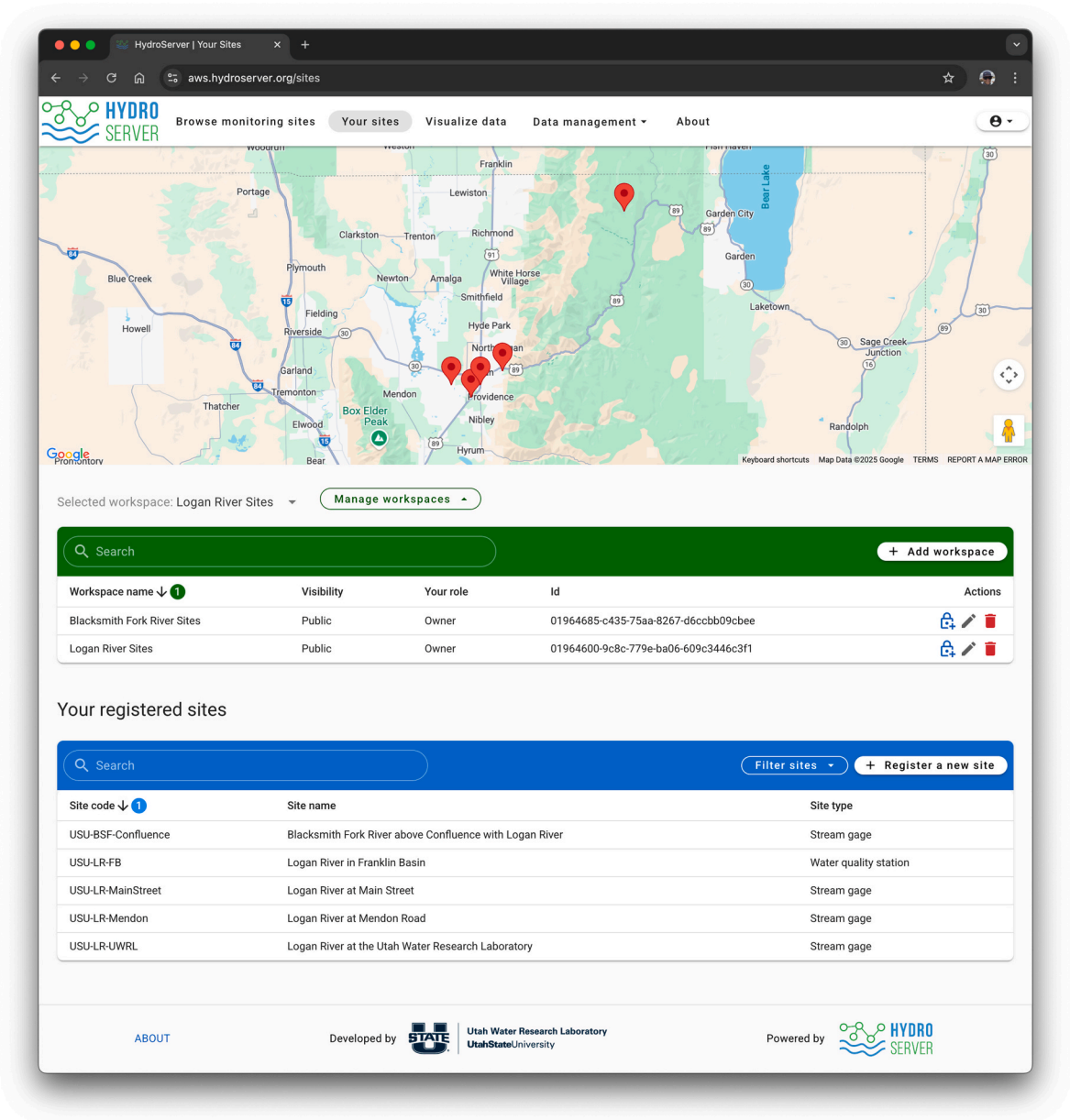
##### 4.3.2. Your sites page

The “Your sites” page (Fig. 4) provides a map and list of monitoring sites that the user has created or that they have been given access to by another user. Sites are organized on this page within the workspace to which they belong, with access control to sites being controlled at the workspace level. Users can select a workspace to view using a drop-down box on the page, and the list of registered sites displayed at the bottom of the page contains all sites within the selected workspace. By



**Fig. 3.** HydroServer’s “Browse Monitoring Sites” page showing individual markers for monitoring sites (red markers), marker clusters for monitoring sites located close together that separate when zoomed in (blue circles with numbers indicating how many sites are clustered), and pop up bubble that provides site metadata and a link to the site landing page when a user clicks on a monitoring site. The scale of the map is determined by the user with dynamic zoom controls provided by the Google Maps component. (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)





**Fig. 4.** HydroServer’s “Your sites” page showing workspace selection and management along with the list of monitoring sites for the selected workspace. The web map on this page defaults to a zoom level that shows all of the user’s registered monitoring sites within the selected workspace. Individual monitoring site locations are shown as clickable red markers, and the scale of the map is controlled by the user using Google Maps’ built in zoom controls. (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

default, the map is automatically zoomed to the spatial extent of the monitoring sites within the workspace, but the map can be zoomed to any scale desired by the user using Google Maps’ built in zoom controls. A “Manage workspaces” button displays a list of workspaces that the user has access to along with an “Add workspace” button that allows the user to create a new workspace. Specific actions on a workspace include choosing its visibility as either public or private, adding users to a workspace as collaborators, editing the name of the workspace, and deleting the workspace, including its contents. Workspaces have a primary owner (initially set as the user that created the workspace), and ownership of a workspace can be transferred from one user to another by the primary owner. Collaborators can be added to a workspace with Editor or Viewer roles. Editors have access to create and manage sites within a workspace, whereas viewers have read-only access. Only the workspace owner can change its name, toggle its visibility as public or private, or delete it. Only public workspaces are accessible to general

HydroServer users via the “Browse monitoring sites” page. The access control settings created for the workspace are honored by HydroServer’s APIs.

Users can register new monitoring sites on the “Your sites” page by clicking a “Register a new site” button and then filling out a form with required and optional metadata for the site, including photographs of the site location. Where users have a large number of monitoring sites within a workspace, tags can be assigned as flexible metadata elements and then used to filter the view of monitoring sites to only those that have a particular tag. Sites can also be filtered by name using a search box at the top of the site list.

Clicking on a site on the “Your sites” page opens the landing page for that site (Fig. 5), which displays metadata about the monitoring site and the list of datastreams associated with the site for all users with view permissions. Users with edit permission can control the list of datastreams for a monitoring site on its landing page. An “Add new



Fig. 5. HydroServer’s monitoring site landing page showing site location map, site metadata, site photos, and available datastreams (table at bottom), including sparkline plots and latest observations. Sparkline plots show the last 72 h of data for each datastream without axes or coordinates to show recent trends in observations.

datastream” button opens a form for the user to specify the sensor used to create the datastream, the observed property the datastream represents, the units associated with the datastream, and the processing level. Additional datastream attributes specified by the user include the sampled medium, status, aggregation statistic, no data value, time aggregation interval, and intended time spacing. The name and description

of the datastream can either be entered by the user or can be auto generated on the form using the other metadata elements. Once created, a datastream is assigned a unique identifier which can then be used to load or retrieve data via HydroServer’s APIs.

The sparkline plots on the site landing page show the most recent 72 h of data for each datastream, and the color of the plot indicates whether

data have been recently updated. Sparkline plots are automatically scaled to the minimum and maximum data values within the 72 h period to provide an “at-a-glance” indication of recent data behavior without axes or coordinates. In the “Actions” column for each datastream, a user with edit permission can specify additional granularity to the access control for each datastream’s metadata and data values. Datastreams can be completely private, private data with public metadata, or completely public. Clicking the actions button for a datastream allows users with view permission to launch the datastream into the “Visualize data” page or download a CSV file containing the datastream, and users with edit permissions can edit the datastream’s metadata or delete the datastream.

#### 4.3.3. Visualize data page

The “Visualize data” page (Fig. 6) provides visualization capabilities for datastreams stored in HydroServer. It is meant to be a screening level tool for monitoring incoming data and screening level data analysis. Users select a list of up to five datastreams for plotting from the list of datastreams at the bottom of the page. Filters for monitoring site, observed property, and processing level in the left column of the page allow users to limit the datastreams shown in the table at the bottom to make it easier to select the data the user wishes to plot, and the filters panel is collapsible to provide additional space for the plot. Plot controls enable panning, zooming, and switching between the plot view and a view of summary statistics for the selected datastreams. Users can also

use a plot control to export the current plot to a png image. A time slider control at the bottom of the plot allows controlling the time window displayed, and users can limit the time range of data selected for plotting by clicking on buttons for last year, last month, or last week. Users can also specify a custom date range for selected data. The x and y-axes are automatically scaled to match the temporal extent and y-axis range of the data selected for plotting. A button to “Copy State as URL” is provided near the center of the page, which generates a URL that can be used to launch HydroServer in a web browser with the “Visualize data” page activated with the current data selections and plot shown. This is useful where a user wants to share a particular view of data.

#### 4.3.4. Data management pages

HydroServer’s “Data management” pages are intended to help users manage metadata for monitoring sites and datastreams within a workspace as well as for setting up and managing automated data loading using HydroServer’s Streaming Data Loader software. In this section we describe the “Manage metadata” page. The “Job orchestration” page is described in Section 4.4 that describes HydroServer’s Streaming Data Loader. Because it can be tedious for users to repeatedly enter metadata when creating new datastreams, we created the “Manage metadata” page to enable users to create and modify metadata for sensors, observed properties, processing levels, units, and result qualifiers within a workspace. Once instances of these entities have been created, they are stored in HydroServer’s database and can then be chosen from drop

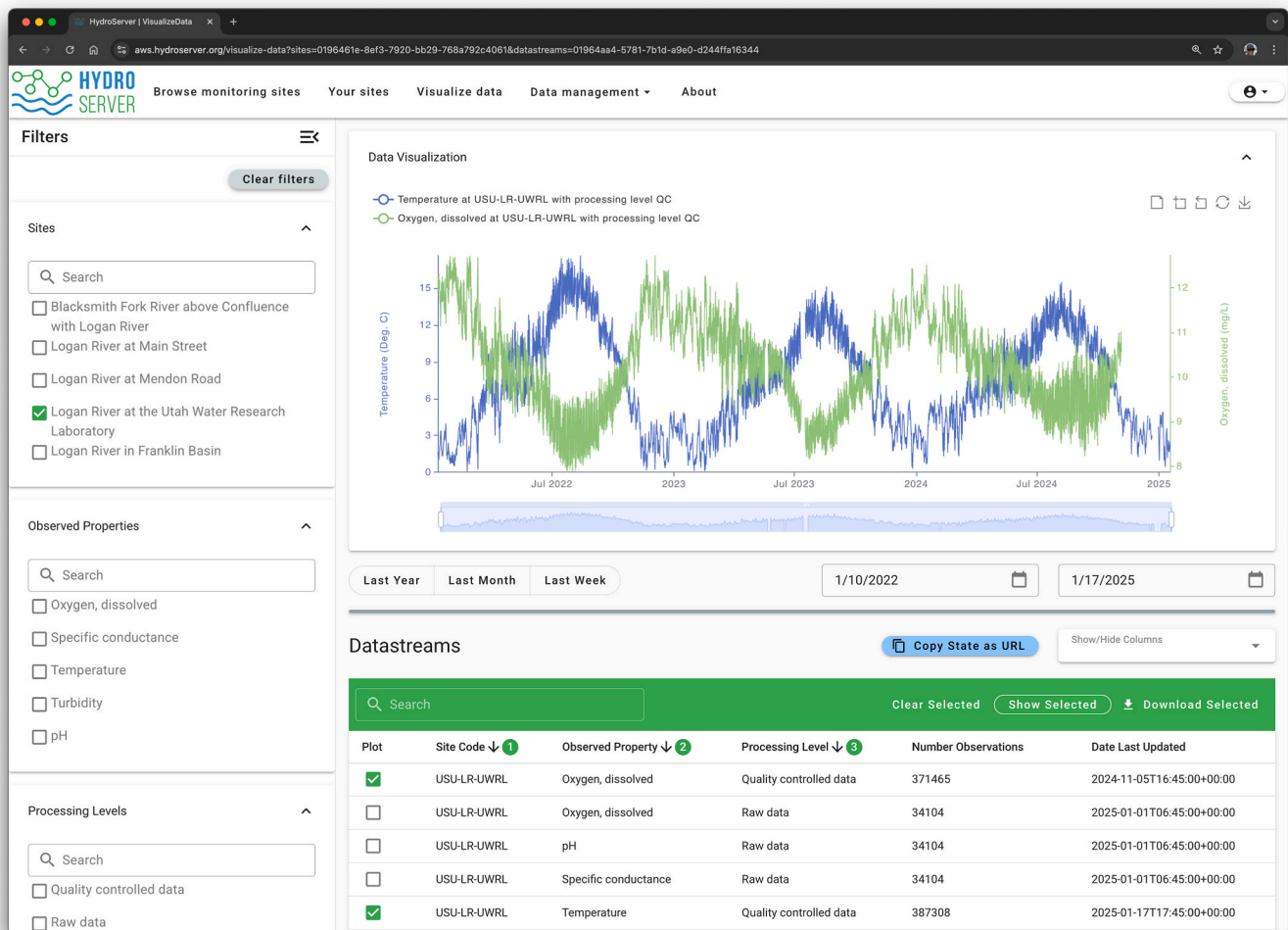


Fig. 6. HydroServer’s “Visualize data” page showing datastream selection in the table at the bottom, datastream filters for site, observed properties, and processing levels in the left column, and the plot window, date selection controls, and plot zoom controls at top right.

down lists when creating new datastreams that use those entities, which significantly reduces the amount of time required to create metadata for each new datastream. On the tabs on the “Manage metadata” page, users can click on a button to add a new instance of each entity (e.g., “Add new sensor”) and then specify the required and optional metadata elements for the new entity instance. Users can also modify and delete entity instances from these pages. While the “Manage metadata” page can help streamline metadata creation and management, it may still be tedious for users to enter metadata for a large number of monitoring sites and datastreams when first populating a HydroServer instance. In such cases, all of the monitoring site and datastream metadata can be created by scripting using HydroServer’s APIs and the *hydroserverpy* client package.

All user-created metadata instances (e.g., sensors, observed properties, etc.) are specific to the workspace within which they are created. This enables different sets of users to control their own metadata. However, there are use cases where users may wish to standardize their metadata across multiple workspaces within a single HydroServer instance. This can be done by defining system-level metadata. Administrators of a HydroServer can define system-level metadata that can then be used across all workspaces in a HydroServer instance. We used the Django Admin interface to create this functionality given that only a small number of administrator users would interact with it and because it would not be needed for all HydroServer instances. System metadata are displayed within a table below the workspace-specific metadata on the Manage metadata page.

#### 4.3.5. Customizing the data management web app

The data management web app can be customized to meet an organization’s needs without requiring significant code modifications. Customization is facilitated through a dedicated config directory within the frontend repository, which exposes variables and pages that users can update. To deploy and customize the app, users can fork the “hydroserver-ops” GitHub repository, which contains the configuration settings and customizable components. By editing the variables and content within this repository, users can tailor aspects such as the default zoom level and extent of the map on the “Browse monitoring sites” page to highlight specific groups of monitoring sites. They can also choose to add a customizable landing page to include organizational information or details about the purpose of their HydroServer instance. Additionally, the “About” page can be personalized with information about the organization or data collection programs served by the HydroServer instance, including contact information.

This technical approach streamlines the customization process by eliminating the need to fork the main “hydroserver-data-management-app” GitHub repository. It allows organizations to maintain their unique configurations separately while benefiting from updates and improvements to the core application. By abstracting customization into a separate GitHub repository, HydroServer ensures that deploying a tailored instance is straightforward and maintainable using available tools that work with GitHub to help automate deployments.

#### 4.4. Streaming Data Loader

The Streaming Data Loader (SDL) is a desktop app that can be used to automatically load data into HydroServer from CSV files containing sensor data (e.g., CSV files output by a datalogger). The SDL can be run on the latest Windows and Mac operating systems and consists of a simple setup interface for signing in with HydroServer credentials, creating a unique name for the SDL instance, and for viewing log information along with the executable data loader that runs silently in the background and can be started automatically and run as a service. We chose to develop a desktop app to ensure that data could be loaded automatically to a HydroServer instance from whatever computer the data resides on.

The SDL was written using Python for consistency with the rest of HydroServer’s software development and uses PySide6 (<https://pypi.org/project/PySide6/>) for a minimal graphical user interface (GUI)

and APScheduler (<https://pypi.org/project/APScheduler/>) for task scheduling. We developed builds for MacOS and Windows operating systems using PyInstaller (<https://pypi.org/project/pyinstaller/>), and a build for Debian distributions of Linux is also available as an executable file. We chose PySide6 because it has cross platform compatibility and a permissive user license. APScheduler was chosen because it provides existing functionality for scheduling the execution of code at specific times or intervals, which is required functionality for the SDL. PyInstaller was used because it simplified distribution and deployment for the SDL app.

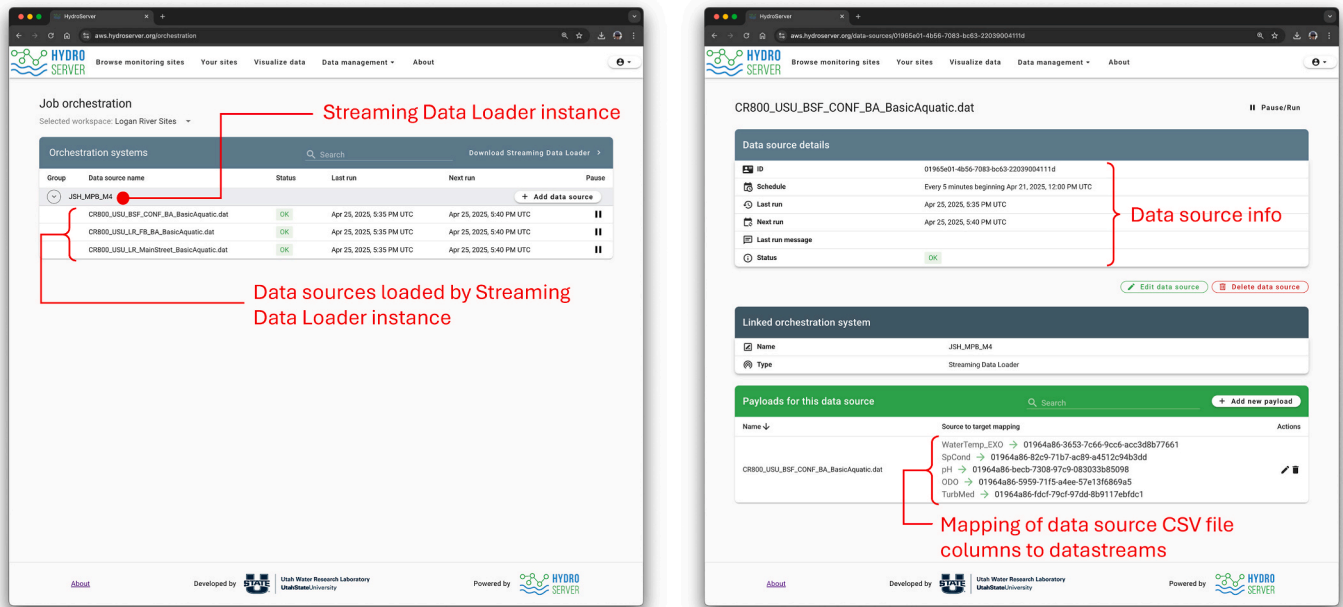
Configuration of which CSV files to load data from and which datastreams to load data to is done in the “Job orchestration” page of the HydroServer data management web app (Fig. 7). The SDL retrieves all information about which data to load from HydroServer’s data management API (Fig. 8). There can be one or many instances of the SDL set up to load data to a HydroServer instance, and each SDL instance must have Internet access to use HydroServer’s APIs along with access to the files containing data to be loaded, either in the local file system or on the Internet via HTTP.

When configuring data loading tasks, a CSV file containing data to be loaded to HydroServer is called a “data source”, and a computer running an instance of the SDL is called an “orchestration system”. The generic name “orchestration system” was chosen because we intend to add other data loading job orchestration systems to HydroServer such that the SDL is only one software mechanism for loading data. An orchestration system can load data from any number of data sources. The “Job orchestration” page (Fig. 7) allows users to see which orchestration systems have been configured to load data to that HydroServer instance. Once an instance of the SDL has been installed and connected to HydroServer it will automatically show up on this page. Users can also download the installer for the SDL from this page. Users can set up and configure data sources under an orchestration system by clicking the “Add Data Source” button and then for each data source, users must specify a path to the CSV file (either a local path on the computer running the SDL or an HTTP URL to the CSV file), the file’s delimiter, the rows on which the column headers appear and on which the data values begin, the name or index of the column containing the timestamp, a format for the timestamp, and a schedule on which data in the file should be loaded. Users can also set the time zone in which their data are located to ensure that the SDL correctly converts timestamps in the file to UTC, which is the standard for data storage in HydroServer. Because orchestration systems must be authorized to load data for monitoring sites, both orchestration systems and data sources are managed within a workspace.

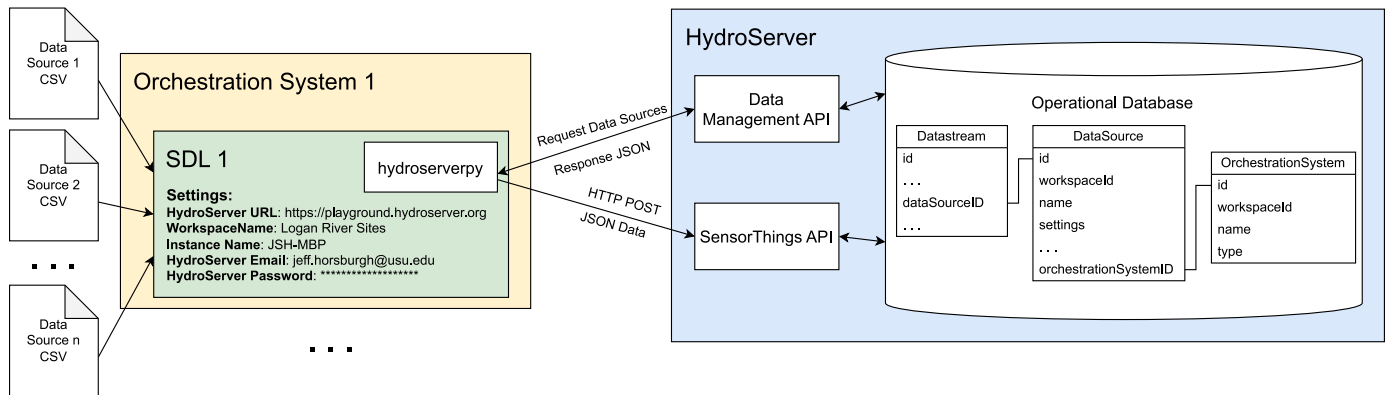
Once configuration of a data source is complete, the columns in the CSV file can be mapped to specific datastreams in the HydroServer database by defining a “payload”. A payload defines a specific set of data to be loaded into HydroServer by mapping the name or index of a column in the CSV file to a specific datastream in HydroServer. The SDL assumes that all of the data columns within a single CSV file share a single column containing date/time information for data values. Creating a payload for a data source requires specifying the name of a data column in the CSV file (e.g., “WaterTemp\_EXO”) and choosing an existing datastream into which data will be loaded, where datastreams are mapped using their identifier (e.g., “01964a86-3653-7c66-9cc6-acc3d8b77661”). Configuration information for orchestration systems, data sources, and mappings of the data content within data sources to datastreams are stored in the OrchestrationSystem, DataSource, and Datastream tables within HydroServer’s database.

Once data sources have been defined and columns within data sources have been linked to datastreams, the SDL will retrieve this information from HydroServer’s data management API when it starts and will proceed to load data from any data sources that have been set up. The data flow for loading data using SDL is shown in Fig. 8. The SDL runs in the background on the machine where it is installed and can be set to





**Fig. 7.** HydroServer Job orchestration page (left). For the “Logan River Sites” workspace, a single instance of the HydroServer Streaming Data Loader called “JSH\_MBP\_M4” is connected and set to load data from three data sources that are datalogger output files from monitoring stations in the LRO. Details for a data source (right) show schedule info, the linked orchestration system, and the mapping of columns in the source CSV file to HydroServer datastreams into which data will be loaded.



**Fig. 8.** HydroServer SDL data flow. An instance of the SDL (green box at left) is registered as an orchestration system (yellow box) with HydroServer (blue box). Each SDL instance can load data from one or more data source CSV files (shown at far left). The SDL uses the *hydroserverpy* Python package to retrieve configuration information from HydroServer’s database and to load data through HydroServer’s APIs (middle section of the figure with *hydroserverpy* at middle left and HydroServer’s APIs at middle right). (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

run on startup to ensure that data loading is not interrupted by machine reboots. On the user-defined schedule, the SDL will parse the data source file, detect any new data that has not already been loaded into HydroServer, and then load new data through HydroServer’s STA. The “Job orchestration” page provides information regarding the status of all data sources that have been configured (Fig. 7, left). Orchestration systems push status updates to HydroServer whenever they try to load data, which enables display of the current status of each data source.

#### 4.5. Archival to HydroShare

Archival to HydroShare is generally done for each individual monitoring site and can be executed via the landing page for that site. Before using HydroShare archival, users must first have a HydroShare user account and must authorize the HydroServer data management web app to use their HydroShare user account to write data to HydroShare. Authorization is performed by accessing the user’s profile in the data

management web app and clicking a button to “Connect to HydroShare”. OAuth 2.0 is then used to authorize the data management web app to access HydroShare on the user’s behalf. Users then click a “Configure HydroShare Archival” button on a monitoring site’s landing page to configure this option via a web form and then specify whether they want to create a new resource in HydroShare or archive data to an existing HydroShare resource (Fig. 9). When using an existing resource, users need only provide the URL to the resource’s landing page, a name for the folder in which archive files will be stored within the resource, and the list of datastreams on this form with all datastreams available at that monitoring site, after which the user can select specific datastreams to archive if they do not want to include all of them. When a user chooses to create a new HydroShare resource, they must supply a title, abstract, keywords, and sharing status for the resource so that HydroServer can create the resource via HydroShare’s REST API before loading data. Metadata fields are pre-populated by HydroServer but can be modified

The screenshot shows a web form titled "Create HydroShare Archival Resource". At the top, there are two radio buttons: "Create New Resource" (selected) and "Link Site to Existing Resource". Below this, the form is divided into several sections:

- Resource Title:** A text input field containing "HydroServer Archive: Blacksmith Fork River above Confluence with Logan River".
- Resource Abstract:** A text area containing a paragraph about the HydroShare resource serving as an archive for monitoring data at Blacksmith Fork River.
- Resource Keywords:** A row of three tags: "HydroServer Site Archive", "Liquid aqueous", and "Stream gage".
- Resource Folder Name:** A text input field containing "HydroServer".
- Included Datastreams:** A list of five datastreams, each with a description and a status icon (e.g., "A datastream of Oxygen, dissolved at Blacksmith Fork River...").
- Resource Sharing Status:** A toggle switch set to "Public".

Red annotations with arrows point to specific parts of the form:

- A bracket on the right side of the "Resource Title", "Resource Abstract", and "Resource Keywords" sections is labeled "HydroShare resource metadata (Auto generated)".
- An arrow points from the "Resource Folder Name" field to the text "Folder in which to copy archive files".
- A bracket on the right side of the "Included Datastreams" list is labeled "Datastreams to archive".
- An arrow points from the "Resource Sharing Status" toggle to the text "HydroShare resource sharing status".

At the bottom right of the form are "Close" and "Create" buttons.

**Fig. 9.** Archival to HydroShare configuration. New or existing resource selection is shown at top. HydroShare resource metadata elements (title, abstract, and keywords) shown in the middle, and specific archival settings, including the folder name, list of datastreams to be archived, and sharing status for the resource shown at the bottom.

by the user if desired.

One data file containing the entire period of record is exported per selected datastream to the designated folder within the HydroServer resource created or selected by the user using HydroShare's REST API. Data files are written in a CSV format with a descriptive header containing metadata about the datastream from the operational database, and then timestamps, data values, and qualifying comments are written as comma separated values to the file. Where a user wishes to archive data for more than one monitoring site to the same HydroShare resource, they can set up the resource in HydroShare and then archive to that resource from each individual monitoring site using a different folder name for each site. Archival is currently run when a user requests it. Archival configuration settings are stored in HydroServer's "Data-Archive" table so users can run the archival job whenever needed. Future functionality is planned for enabling scheduled archival to HydroShare using a job orchestration system.

#### 4.6. Deploying HydroServer

We designed HydroServer so that it can be deployed by a data collection organization for managing their own data (as we do for the LRO) or by an organization that hosts an instance as a service for others who simply want to use the software. Thus, we anticipate that there will eventually be many deployed instances of HydroServer. There are several technical options for deploying HydroServer. The data management web app and web service interfaces must be installed on a web server, and HydroServer's operational database requires a database server running PostgreSQL. Servers may be provisioned as local physical or virtual machines, or they may be provisioned as virtual machines within a private or commercial cloud platform. HydroServer can also be deployed using infrastructure as a service within commercial cloud platforms. The choice of how to deploy HydroServer depends on an organization's needs, the volume of data to be managed, and the volume of data use. Commercial cloud deployments provide the greatest flexibility, with the ability to scale resources as the volume of data and use grow over time.

We developed detailed instructions within HydroServer's documentation for deploying HydroServer using Amazon Web Services (AWS) and Google Cloud Platform (GCP), two of the most widely used

commercial cloud platforms. Terraform (<https://www.terraform.io/>), an infrastructure-as-code (IaC) tool, is used for infrastructure provisioning and management because it helps streamline the deployment process and minimize errors. Additionally, we built a continuous integration/continuous deployment (CI/CD) pipeline with GitHub Actions workflows to automate tasks such as validating Terraform configurations, applying infrastructure changes, and deploying updates to the HydroServer application. We chose GitHub Actions because HydroServer's code is already hosted in GitHub, making the deployment actions more seamless. The Terraform IaC scripts and GitHub Actions workflows are included in a GitHub repository (<https://github.com/hydroserver2/hydroserver-ops>) that individuals or organizations can fork to manage their own HydroServer deployments.

#### 5. Discussion

HydroServer provides an open-source, standards-based software platform for environmental sensor data management. It meets many of the data collection, communication, storage, management, and sharing needs of groups like the LRO who collect large volumes of sensor data and need cyberinfrastructure to support day-to-day operations. Through our deployment and testing of the HydroServer instance for the LRO, we have verified that the functional use cases and requirements listed in Section 2 have been met. Based on success with the LRO, we are also now using HydroServer as a platform for modernizing the state of Utah's water use data infrastructure in collaboration with Utah's Division of Water Rights (UDWRi). UDWRi operates or retrieves data from more than 1500 continuous data collection stations throughout Utah, with observations representing flows at agricultural diversions, pumping rates at groundwater wells, water levels and storage values for reservoirs, and discharge at streamflow gages. These data are used by UDWRi for water distribution accounting according to Utah's legal water rights system, with HydroServer serving as a data management platform and the source of data for operational decision making.

HydroServer's use of the OGC STA standard attracted the attention of the World Meteorological Organization (WMO) who support development of interoperable hydrologic monitoring and forecast systems in several areas of the world. WMO is working to improve accessibility of hydrologic data for integration with their Integrated Global Observing

System (WIGOS), WMO Information System (WIS), and the WMO Hydrologic Observing System (WHOS, <https://wmo.int/activities/wmo-hydrological-observing-system-whos>), which is the hydrologic component of WIGOS. HydroServer fits with the architecture of WHOS by enabling data providers to share hydrologic data via standards-based web service interfaces and data encodings (using OGC's STA) which can be integrated with the WHOS Broker, a software system that provides standardization, discovery, and access to data. HydroServer is interoperable as part of this larger system because of its adoption of the OGC STA standard. HydroServer is also under consideration by the GEOGloWS Initiative (<https://www.geogloWS.org/>) as a platform for enabling hydrologic measurement services in various countries to make their hydrologic observations available for use in local bias correction of GEOGloWS' global streamflow forecast products (Hales et al., 2022, 2023). In both cases, HydroServer serves as a viable software system for hydrologic data collection services in countries that either cannot afford or choose not to use available commercial software systems, and HydroServer's access control enables data collection organizations to choose which data become publicly available. HydroServer's flexibility for deployment in the commercial cloud or using local infrastructure also enables organizations, regardless of where they are located, to choose a deployment that best meets their needs for performance and scalability. Extending HydroServer deployment for additional organizations, including UDWRi, WMO, and GEOGloWS verifies that HydroServer's architecture, which is modeled after the generalized HIS architecture presented by Spackman Jones and Horsburgh (2025), is generalizable and meets the requirements and data use cases for multiple organizations.

HydroServer's APIs, Python client package, and Streaming Data Loader enable interoperability and automation. There are multiple pathways for loading data into a HydroServer instance, which opens pathways for interoperability with different sensors, dataloggers, and proprietary software systems. We designed these mechanisms to ensure that data could be loaded into HydroServer regardless of how the data is collected and without replacing software that data managers may already be using. For example, HydroServer's SDL can be set up to load data from CSV files created by Campbell Scientific's Loggernet software after retrieving data from remote monitoring stations. This is our primary mechanism for automated data loading for the LRO. The combination of HydroServer's data management web app and APIs enables a HydroServer instance to be managed by a person through a graphical user interface or by scripting and automation through HydroServer's APIs. This flexibility also ensures that HydroServer does not become a stovepipe that limits integration with other software platforms used for data visualization or analysis – e.g., users can download data from the data management web app, HydroServer's Python client can be used to pull data into a Python analytical environment for visualization and analysis, or developers could use HydroServer's API to build additional apps for visualizing or presenting data.

As with the original version of HydroServer, the modernized version reported here could participate within a services-oriented architecture that provides catalog services and data search and discovery across multiple HydroServer instances. OGC's STA provides a standard set of endpoints that could be used to harvest metadata about available monitoring sites and datastreams from HydroServer instances – as did the original WaterOneFlow web services. Doing so is beyond the scope of this paper; however, the STA data model we adapted for HydroServer would provide a useful starting point for developing catalog and discovery services across multiple HydroServer instances.

While HydroServer may meet the needs of many groups who collect and manage environmental sensor data, there are limitations that are challenges for wide adoption. For advanced research groups with software development expertise who want to customize HydroServer to meet their needs, our specific technology choices may or may not be compatible with their expertise or other systems they maintain. Other groups may lack the expertise to deploy an instance of HydroServer for

their own use, leaving an opportunity for potential hosting of HydroServer instances as a service for these types of users. Additionally, since HydroServer is relatively new, we are still uncovering bugs and fixing them as we use HydroServer in operational contexts. Issues with performance and scalability, particularly for instances with large data volumes, can be solved with appropriate configurations within cloud deployment environments, but, as with any cloud-based software deployment, expertise is required to “right-size” cloud resources to minimize costs while meeting performance objectives. This expertise may likewise be beyond some groups' existing capabilities. Although these are significant considerations, they are not all specific to HydroServer and can be overcome where an organization needs to invest in a data management solution.

Future HydroServer development will include our current work on developing HydroServer's data QC software app. The QC app has strong potential to enhance the reproducibility and quality of data QC workflows by providing a consistent user interface, data correction tools/methods, and ability to record the sequence of edits/modifications made in a QC workflow. We are also considering integration of the Message Queuing Telemetry Transport (MQTT) extension of the OGC STA into HydroServer's STA implementation. This would enhance HydroServer's ability to meet additional IoT sensing use cases by enabling resource-constrained IoT devices to send data to a HydroServer instance using the MQTT protocol instead of HydroServer STA's current HTTP REST interface. Message broker technologies like MQTT can facilitate fast, reliable, scalable, and flexible communication between different parts of a distributed sensing system, which may open new opportunities for developing smarter monitoring stations that are more intelligent, expandable, interoperable, and capable of delivering situational awareness through using asynchronous communication that does not require direct connections or concurrent availability. In this context, HydroServer could act as both a subscriber to data feeds from sensing nodes and as a publisher to which sensing nodes could subscribe. There is also opportunity to integrate monitoring sites, their geospatial locations, and their observational data stored in a HydroServer instance as points of interest registered within national-scale “HydroFabric” datasets via systems like GeoConnex (Doyle and Onda, 2023) or the HydroFabrics under operational development and use by the USGS and the National Oceanic and Atmospheric Administration for national scale modeling efforts (e.g., Bock et al., 2022; Blodgett et al., 2023).

## 6. Conclusions

Software is required for effective collection, communication, storage, management, and sharing of data from in situ environmental sensors. HydroServer provides a software platform designed to meet these needs for organizations that operate sensors for environmental monitoring. HydroServer's open source code and adoption of the open OGC STA standard have positioned it as a highly interoperable system for environmental sensor data management, capable of supporting both local and international data collection and sharing initiatives. Successful deployment of HydroServer for the LRO and UDWRi's water use data infrastructure demonstrates its capacity to manage large volumes of sensor data. HydroServer's flexibility in deployment, whether through commercial cloud platforms or local infrastructure, ensures that it can be tailored and scaled to meet the specific needs of a wide range of organizations. We anticipate that HydroServer will be useful for many different organizations looking for an accessible and open source software solution for managing data from environmental sensors.

Integration of HydroServer with international initiatives, such as the World Meteorological Organization's WHOS and the GEOGloWS Initiative, highlights its potential for broader application in international data sharing. By providing data managers with a robust set of APIs and client tools, HydroServer fosters interoperability with existing systems. This ensures that organizations can integrate data from a variety of sensors, dataloggers, and systems using their existing and/or preferred

software tools while benefiting from HydroServer's centralized data management and sharing capabilities. HydroServer's support for automated data loading through the SDL and the ability to script metadata creation through HydroServer's Python client package also enhance the efficiency of data handling, particularly for larger scale operations. Archival to HydroShare provides an off-ramp at the end of a project or data collection initiative so that an organization need not maintain an active HydroServer instance in perpetuity.

Looking forward, HydroServer's development will focus on expanding its capabilities, particularly in the areas of data QC and potential integration of the MQTT protocol STA extension to expand HydroServer's capabilities to support data collected by IoT devices. These advances will enhance HydroServer's utility for a broader range of environmental monitoring use cases, including those requiring real-time data streaming from resource-constrained IoT devices. Additionally, potential integration with national scale hydrofabrics presents an opportunity for HydroServer to contribute to larger scale hydrologic modeling and analysis efforts, further cementing its role as a critical tool for both local and global water resource management.

### CRedit authorship contribution statement

**Jeffery S. Horsburgh:** Writing – original draft, Resources, Methodology, Investigation, Funding acquisition, Conceptualization. **Kenneth Lippold:** Writing – review & editing, Software, Methodology, Investigation. **Daniel L. Slauch:** Writing – review & editing, Software, Methodology, Investigation. **Maurier Ramirez:** Writing – review & editing,

Software, Methodology, Investigation.

### Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Jeffery S. Horsburgh reports financial support was provided by National Oceanic and Atmospheric Administration. Jeffery S. Horsburgh reports financial support was provided by State of Utah Division of Water Rights. Jeffery S. Horsburgh reports financial support was provided by World Meteorological Organization. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

### Acknowledgments

This research was supported by the Cooperative Institute for Research to Operations in Hydrology (CIROH) with funding under award NA22NWS4320003 from the National Oceanic and Atmospheric Administration (NOAA) Cooperative Institute Program. The statements, findings, conclusions, and recommendations are those of the authors and do not necessarily reflect the views of NOAA. Additional funding and support were provided by the Utah Water Research Laboratory at Utah State University, the State of Utah Division of Water Rights, and the World Meteorological Organization.

## Appendix A

### Summary of *hydroserverpy* Classes

This Appendix provides a summary of the *hydroserverpy* HydroServer Python client package classes, including a description of each class along with their public properties and methods. This appendix focuses on classes, properties, and methods accessed directly by users (i.e., private properties and methods have been omitted). More complete documentation is available via *hydroserverpy*'s documentation hosted in GitHub <https://hydroserver2.github.io/hydroserverpy/>. Table A.1 provides a high-level description of each of the *hydroserverpy* classes, after which the following tables document the public properties and methods available in each class.

**Table A.1**  
*hydroserverpy* class descriptions.

| Class Name                 | Description                                                                                                                     |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Base Classes               |                                                                                                                                 |
| HydroServer                | Connects to a HydroServer instance and handles user authentication. Enables interaction with HydroServer's Data Management API. |
| HydroServerBaseModel       | The base class for HydroServer entities that provides methods for data manipulation and persistence.                            |
| EndpointService            | The base class for HydroServer API endpoints that provides common methods for interacting with data.                            |
| EndpointServiceClasses     |                                                                                                                                 |
| WorkspaceService           | Enables retrieval and management of HydroServer workspaces.                                                                     |
| OrchestrationSystemService | Enables retrieval and management of HydroServer orchestration systems.                                                          |
| DataSourceService          | Enables retrieval and management of HydroServer data sources.                                                                   |
| DataArchiveService         | Enables retrieval and management of HydroServer data archives.                                                                  |
| DatastreamService          | Enables retrieval and management of HydroServer datastreams.                                                                    |
| ObservedPropertyService    | Enables retrieval and management of HydroServer observed properties.                                                            |
| ProcessingLevelService     | Enables retrieval and management of HydroServer processing levels.                                                              |
| ResultQualifierService     | Enables retrieval and management of HydroServer result qualifiers.                                                              |
| SensorService              | Enables retrieval and management of HydroServer sensors.                                                                        |
| ThingService               | Enables retrieval and management of HydroServer things (monitoring sites).                                                      |
| UnitService                | Enables retrieval and management of HydroServer units.                                                                          |
| ModelClasses               |                                                                                                                                 |
| Workspace                  | Represents an individual workspace in HydroServer.                                                                              |
| Account                    | Represents an individual account in HydroServer.                                                                                |
| Role                       | Represents an individual role in HydroServer.                                                                                   |
| Collaborator               | Represents an individual collaborator in HydroServer.                                                                           |
| OrchestrationSystem        | Represents an individual orchestration system in HydroServer.                                                                   |
| DataArchive                | Represents an individual data archive in HydroServer.                                                                           |
| DataSource                 | Represents an individual data source in HydroServer.                                                                            |
| Datastream                 | Represents an individual datastream in HydroServer.                                                                             |
| ObservedProperty           | Represents an individual observed property in HydroServer.                                                                      |

(continued on next page)



**Table A.1** (continued)

| Class Name      | Description                                               |
|-----------------|-----------------------------------------------------------|
| ProcessingLevel | Represents an individual processing level in HydroServer. |
| ResultQualifier | Represents an individual result qualifier in HydroServer. |
| Sensor          | Represents an individual sensor in HydroServer.           |
| Thing           | Represents an individual thing in HydroServer.            |
| Unit            | Represents an individual unit in HydroServer.             |

**Table A.2**

Properties of the HydroServer class.

| Properties           | Description                                          |
|----------------------|------------------------------------------------------|
| workspaces           | An instance of the WorkspaceService class.           |
| orchestrationsystems | An instance of the OrchestrationSystemService class. |
| dataarchives         | An instance of the DataArchiveService class.         |
| datasources          | An instance of the DataSourceService class.          |
| datastreams          | An instance of the DatastreamService class.          |
| observedproperties   | An instance of the ObservedPropertyService class.    |
| processinglevels     | An instance of the ProcessingLevelService class.     |
| resultqualifiers     | An instance of the ResultQualifierService class.     |
| sensors              | An instance of the SensorService class.              |
| things               | An instance of the ThingService class.               |
| units                | An instance of the UnitService class.                |

**Table A.3**

Methods of the HydroServerBaseModel class.

| Methods | Description                                   |
|---------|-----------------------------------------------|
| refresh | Refreshes/resets the object from HydroServer. |
| save    | Saves changes to object back to HydroServer.  |
| delete  | Deletes the object from HydroServer.          |

**Table A.4**

Methods of the EndpointService class.

| Methods | Description                                          |
|---------|------------------------------------------------------|
| list    | Retrieves a list of objects from HydroServer.        |
| get     | Gets an individual object from HydroServer by ID.    |
| create  | Creates a new object in HydroServer.                 |
| update  | Updates the properties of an object in HydroServer.  |
| delete  | Deletes an individual object from HydroServer by ID. |

**Table A.5**

Methods of the WorkspaceService class.

| Methods                   | Description                                                        |
|---------------------------|--------------------------------------------------------------------|
| list_roles                | Retrieves a list of roles that can be assigned within a workspace. |
| list_collaborators        | Retrieves a list of collaborators associated with a workspace.     |
| add_collaborator          | Adds a new collaborator to a workspace.                            |
| edit_collaborator_role    | Changes the role of a workspace collaborator.                      |
| remove_collaborator       | Removes a collaborator from a workspace.                           |
| transfer_ownership        | Initiates ownership transfer of a workspace to another user.       |
| accept_ownership_transfer | Accepts a pending ownership transfer of a workspace.               |
| cancel_ownership_transfer | Cancels a pending ownership transfer of a workspace.               |

**Table A.6**  
Methods of the DataArchiveService class.

| Methods           | Description                              |
|-------------------|------------------------------------------|
| add_datastream    | Adds a Datastream to a DataArchive.      |
| remove_datastream | Removes a Datastream from a DataArchive. |

**Table A.7**  
Methods of the DataSourceService class.

| Methods           | Description                             |
|-------------------|-----------------------------------------|
| add_datastream    | Adds a Datastream to a DataSource.      |
| remove_datastream | Removes a Datastream from a DataSource. |

**Table A.8**  
Methods of the DatastreamService class.

| Methods           | Description                                                                        |
|-------------------|------------------------------------------------------------------------------------|
| get_observations  | Returns a pandas DataFrame of observations belonging to a datastream given the ID. |
| load_observations | Uploads a pandas DataFrame of observations to HydroServer given a datastream ID.   |

**Table A.9**  
Methods of the ThingService class.

| Methods      | Description                 |
|--------------|-----------------------------|
| add_tag      | Adds a new tag for a Thing. |
| update_tag   | Updates a tag of a Thing.   |
| delete_tag   | Deletes a tag of a Thing.   |
| add_photo    | Adds a photo of a Thing.    |
| delete_photo | Deletes a photo of a Thing. |

**Table A.10**  
Properties and methods of the Workspace class.

| Properties                | Description                                                                 |
|---------------------------|-----------------------------------------------------------------------------|
| roles                     | A list of the roles that can be assigned to collaborators of the workspace. |
| collaborators             | A list of the workspace collaborators.                                      |
| things                    | A list of the things belonging to the workspace.                            |
| observedproperties        | A list of the observed properties belonging to the workspace.               |
| processinglevels          | A list of the processing levels belonging to the workspace.                 |
| resultqualifiers          | A list of the result qualifiers belonging to the workspace.                 |
| units                     | A list of the units belonging to the workspace.                             |
| sensors                   | A list of the sensors belonging to the workspace.                           |
| datastreams               | A list of the datastreams belonging to the workspace.                       |
| orchestrationsystems      | A list of the orchestration systems belonging to the workspace.             |
| datasources               | A list of the data sources belonging to the workspace.                      |
| dataarchives              | A list of the data archives belonging to the workspace.                     |
| Methods                   | Description                                                                 |
| add_collaborator          | Adds a new collaborator to the workspace.                                   |
| edit_collaborator_role    | Edits the role of a workspace collaborator.                                 |
| remove_collaborator       | Removes a collaborator from the workspace.                                  |
| transfer_ownership        | Initiates ownership transfer of the workspace to another user.              |
| accept_ownership_transfer | Accepts a pending ownership transfer of the workspace.                      |
| cancel_ownership_transfer | Cancels a pending ownership transfer of the workspace.                      |

**Table A.11**  
Methods of the Collaborator class.

| Methods   | Description                                   |
|-----------|-----------------------------------------------|
| edit_role | Edits the role of the collaborator.           |
| remove    | Removes this collaborator from the workspace. |

**Table A.12**  
Properties of the OrchestrationSystem class.

| Properties   | Description                                            |
|--------------|--------------------------------------------------------|
| workspace    | The workspace the orchestration system belongs to.     |
| datasources  | The data sources managed by the orchestration system.  |
| dataarchives | The data archives managed by the orchestration system. |

**Table A.13**  
Properties and methods of the DataSource class.

| Properties           | Description                                                                                       |
|----------------------|---------------------------------------------------------------------------------------------------|
| workspace            | The workspace the data source belongs to.                                                         |
| orchestration_system | The orchestration system that manages the data source.                                            |
| datastreams          | The datastreams associated with the data source.                                                  |
| Methods              | Description                                                                                       |
| add_datastream       | Adds a datastream to the data source.                                                             |
| remove_datastream    | Removes a datastream from the data source.                                                        |
| load_observations    | Attempts to load observations to HydroServer for all datastreams associated with the data source. |

**Table A.14**  
Properties and methods of the DataArchive class.

| Properties           | Description                                             |
|----------------------|---------------------------------------------------------|
| workspace            | The workspace the data archive belongs to.              |
| orchestration_system | The orchestration system that manages the data archive. |
| datastreams          | The datastreams associated with the data archive.       |
| Methods              | Description                                             |
| add_datastream       | Adds a datastream to the data archive.                  |
| remove_datastream    | Removes a datastream from the data archive.             |

**Table A.15**  
Properties and methods of the Datastream class.

| Properties        | Description                                                            |
|-------------------|------------------------------------------------------------------------|
| workspace         | The workspace the datastream belongs to.                               |
| thing             | The thing (monitoring site) associated with the datastream.            |
| data_source       | The data source of observations for the datastream.                    |
| observed_property | The observed property associated with the datastream.                  |
| processing_level  | The processing level associated with the datastream.                   |
| unit              | The unit associated with the datastream.                               |
| sensor            | The sensor associated with the datastream.                             |
| Methods           | Description                                                            |
| get_observations  | Returns a dataframe of observations belonging to the datastream.       |
| load_observations | Uploads a dataframe of observations to HydroServer for the datastream. |

**Table A.16**  
Properties and methods of the Thing class.

| Properties   | Description                                                                                    |
|--------------|------------------------------------------------------------------------------------------------|
| workspace    | The workspace the thing belongs to.                                                            |
| datastreams  | The datastreams associated with the thing (monitoring site).                                   |
| tags         | The tags associated with the thing.                                                            |
| photos       | Links to photos associated with the thing.                                                     |
| archive      | Link to a HydroShare resource containing archived data from the thing if one has been created. |
| Methods      | Description                                                                                    |
| add_tag      | Adds a new tag to the thing.                                                                   |
| update_tag   | Updates an existing tag of the thing.                                                          |
| delete_tag   | Deletes a tag of the thing.                                                                    |
| add_photo    | Uploads a photo of the thing to HydroServer.                                                   |
| delete_photo | Deletes a photo of the thing from HydroServer.                                                 |

## Data availability

HydroServer's code is open source in GitHub (see Software Availability). For demonstration purposes, we have deployed an instance of HydroServer for the LRO at <https://lro.hydroserver.org> which contains time series data for stations in the Logan River watershed of northern Utah aimed at collecting comprehensive hydrologic data to inform water management decision making and support hydrologic research. LRO data is openly and freely available. We also provide a public testing instance of HydroServer at <https://playground.hydroserver.org> that allows users to create a user account and test HydroServer's functionality within a playground server environment.

## References

- Ames, D.P., Horsburgh, J.S., Cao, Y., Kadlec, J., Whiteaker, T., Valentine, D., 2012. HydroDesktop: web services-based software for hydrologic data discovery, download, visualization, and analysis. *Environ. Model. Software* 37, 146–156. <https://doi.org/10.1016/j.envsoft.2012.03.013>.
- Blodgett, D., Dornblut, I. (Eds.), 2018. OGC Waterml 2: Part 3 – Surface Hydrology Features (Hy Features) – Conceptual Model, Open Geospatial Consortium, pp. 14–111r6. OGC, Version 1.0. <https://docs.ogc.org/is/14-111r6/14-111r6.html>.
- Blodgett, D., Johnson, J.M., Bock, A., 2023. Generating a reference flow network with improved connectivity to support durable data integration and reproducibility in the conterminous US. *Environ. Model. Software* 165, 105726. <https://doi.org/10.1016/j.envsoft.2023.105726>.
- Bock, A.R., Blodgett, D.L., Johnson, J.M., Santiago, M., Wiczorek, M.E., 2022. PROVISIONAL: national hydrologic geospatial fabric reference and derived hydrofabrics. U.S. Geological Survey Data Release. <https://doi.org/10.5066/P9NFPB5S>. USGS ScienceBase.
- Botts, M., Robin, A., Hirschorn, E., 2020. OGC SensorML: Model and XML Encoding Standard, OGC Implementation Standard, Version 2.1, OGC 12-000r2, <https://docs.ogc.org/is/12-000r2/12-000r2.html>.
- Brodaric, B. (Ed.), 2017. OGC Waterml 2: Part 4 – GroundWaterML 2 (GWML2), Open Geospatial Consortium, pp. 16–32. OGC, Version 2.2. <https://docs.ogc.org/is/16-032r2/16-032r2.html>.
- Bröring, A., Stasch, C., Echterhoff, J., 2012. OGC Sensor Observation Service Interface Standard. OGC Implementation Standard, pp. 12–1006. OGC, Version 2.0. <http://www.opengis.net/doc/IS/SOS/2.0>.
- Bumberger, J., Abbrent, M., Brinckmann, N., Hemmen, J., Kunkel, R., Lorenz, C., Lünenschloss, P., Palm, B., Schnicke, T., Schulz, C., van der Schaaf, H., Schäfer, D., 2025. Digital ecosystem for FAIR time series data management in environmental system science. *SoftwareX* 29, 102038. <https://doi.org/10.1016/j.softx.2025.102038>.
- Campo, F., Franco, D., Santos, F.D.C., Blanco-Rodriguez, A., Garcia-Ramirez, A.R., Ratao, G., Hoinaski, L., 2023. Clean - collaborative low-cost environmental and air-quality network. *Environ. Model. Software* 163, 105664. <https://doi.org/10.1016/j.envsoft.2023.105664>.
- Connor, L., Ames, D.P., Gill, R.A., 2013. HydroServer lite as an open source solution for archiving and sharing environmental data for independent university labs. *Ecol. Inform.* 18, 171–177. <https://doi.org/10.1016/j.ecoinf.2013.08.006>.
- Cox, S., 2013. OGC Abstract Specification Geographic Information - Observations and Measurements. OGC Standard Abstract Specification. OGC 10-004r3, Version 2.0. <http://www.opengis.net/doc/as/om/2.0>.
- Diène, B., Rodrigues, J.J.P.C., Diallo, O., Ndoye, E.H.M., Korotaev, V.V., 2020. Data management techniques for internet of things. *Mech. Syst. Signal Process.* 138, 106564. <https://doi.org/10.1016/j.ymssp.2019.106564>.
- Doyle, M., Onda, K., 2023. Internet of water: research and development toward a linked data system and foundational knowledge network for the internet of water: final report. North Carolina Water Resources Research Institute, Report No. 508, UNC-WRRI-508. <https://www.lib.ncsu.edu/resolver/1840.20/41485>.
- Filhol, S., Lefeuvre, P., Ibañez, J.D., Hulth, J., Hudson, S.R., Gallet, J., Schuler, T.V., Burkhard, J.F., 2023. A new approach to meteorological observations on remote polar glaciers using open source, internet of things technologies. *Front. Environ. Sci.* 11, 1085708. <https://doi.org/10.3389/fenvs.2023.1085708>.
- Hales, R.C., Nelson, J.E., Souffront, M., Gutierrez, A.L., Prudhomme, C., Kopp, S., Ames, D.P., Williams, G.P., Jones, N.L., 2022. Advancing global hydrologic modeling with the GEOGloWS ECMWF streamflow service. *J. Flood Risk Manag.*, e12859. <https://doi.org/10.1111/jfr3.12859>.
- Hales, R.C., Williams, G.P., Nelson, J.E., Sowby, R.B., Ames, D.P., Sanchez Lozano, J.L., 2023. Bias correcting discharge simulations from the GEOGloWS global hydrologic model. *J. Hydrol.* 626 (Part B), 130279. <https://doi.org/10.1016/j.jhydrol.2023.130279>.
- Handl, R., Pizzo, M., Zurmuehl, M., 2016. OData JSON format version 4.0 errata 03. <http://docs.oasis-open.org/odata/odata-json-format/v4.0/e/rrata03/os/odata-json-format-v4.0-errata03-os-complete.html,Latestversion>. <http://docs.oasis-open.org/odata/odata-json-format/v4.0/odata-json-format-v4.0.html>.
- Hodson, T.O., DeCicco, L.A., Hariharan, J.A., Stanish, L.F., Black, S., Horsburgh, J.S., 2023. Reproducibility starts at the source: r, python, and julia packages for retrieving USGS hydrologic data. *Water* 15 (24), 4236. <https://doi.org/10.3390/w15244236>.
- Hooper, R., Maidment, D., Helly, J., Kumar, P., Piasecki, M., 2004. CUAHSI Hydrologic Information Systems, In: Proceedings of the 20th International Conference on Interactive Information and Processing Systems (IIPS) for Meteorology, Oceanography and Hydrology, American Meteorological Society, Seattle, WA.
- Horsburgh, J.S., Aufdenkampe, A.K., Mayorga, E., Lehnert, K.A., Hsu, L., Song, L., Jones, A.S., Damiano, S.G., Tarboton, D.G., Valentine, D., Zaslavsky, I., Whitenack, T., 2016. Observations data model 2: a community information model for spatially discrete Earth observations. *Environ. Model. Software* 79, 55–74. <https://doi.org/10.1016/j.envsoft.2016.01.010>.
- Horsburgh, J.S., Lippold, K., Slaugh, D.L., 2025. Adapting OGC's SensorThings API and data model to support data management and sharing for environmental sensors. *Environ. Model. Software* 183, 106241. <https://doi.org/10.1016/j.envsoft.2024.106241>.
- Horsburgh, J.S., Morsy, M.M., Castronova, A.M., Goodall, J.L., Gan, T., Yi, H., Stealey, M. J., Tarboton, D.G., 2015a. HydroShare: sharing diverse environmental data types and models as social objects with application to the hydrology domain. *J. Am. Water Resour. Assoc.* 52 (4), 873–889. <https://doi.org/10.1111/1752-1688.12363>.
- Horsburgh, J.S., Reeder, S.L., Spackman Jones, A., Meline, J., 2015b. Open source software for visualization and quality control of continuous hydrologic and water quality sensor data. *Environ. Model. Software* 70, 32–44. <https://doi.org/10.1016/j.envsoft.2015.04.002>.
- Horsburgh, J.S., Tarboton, D.G., Maidment, D.R., Zaslavsky, I., 2008. A relational model for environmental and water resources data. *Water Resour. Res.* 44 (5). <https://doi.org/10.1029/2007wr006392>.
- Horsburgh, J.S., Tarboton, D.G., Piasecki, M., Maidment, D.R., Zaslavsky, I., Valentine, D., Whitenack, T., 2009. An integrated system for publishing environmental observations data. *Environ. Model. Software* 24, 879–888. <https://doi.org/10.1016/j.envsoft.2009.01.002>.
- Horsburgh, J.S., Tarboton, D.G., Schreuders, K.A.T., Maidment, D.R., Zaslavsky, I., Valentine, D., 2010. HydroServer: a platform for publishing space-time hydrologic datasets. In: Proceedings of the AWRA Spring Specialty Conference on GIS and Water Resources. [https://www.researchgate.net/publication/239521091\\_HydroServer\\_A\\_Platform\\_for\\_Publishing\\_Space-Time\\_Hydrologic\\_Datasets](https://www.researchgate.net/publication/239521091_HydroServer_A_Platform_for_Publishing_Space-Time_Hydrologic_Datasets).
- Jones, A., Aanderud, Z.T., Horsburgh, J.S., Eiriksson, D., Dastrup, D., Cox, C., Jones, S.B., Bowling, D.R., Carlisle, J., Carling, G.T., Baker, M.A., 2017. Designing and implementing a network for sensing water quality and hydrology across Mountain to urban transitions. *J. Am. Water Resour. Assoc.* 1–26. <https://doi.org/10.1111/1752-1688.12557>.
- Liang, S., Khalafbeigi, T., 2019. OGC SensorThings API part 2 - tasking core. In: Open Geospatial Consortium Implementation Standard, 17–079r1. OGC, Version 1.0. <http://docs.opengis.net/is/17-079r1/17-079r1.html>.



- Liang, S., Khalafbeigi, T., van der Schaaf, H., 2021. OGC SensorThings API part 1: sensing version 1.1. Open Geospatial Consortium. Implement. Standard 18–88. OGC, Version 1.1. <https://docs.ogc.org/is/18-088/18-088.html>.
- Neilson, B.T., Tennant, H., Strong, P.A., Horsburgh, J.S., 2021. Detailed streamflow data for understanding hydrologic responses in the logan river observatory. Hydrol. Process., e14268 <https://doi.org/10.1002/hyp.14268>.
- Maidment, D.R., 2008. Bringing water data together. J. Water Resour. Plan. Manag. 134, 95–96. [https://doi.org/10.1061/\(asce\)0733-9496\(2008\)134:2\(95\)](https://doi.org/10.1061/(asce)0733-9496(2008)134:2(95)).
- O'Geen, A., Safeeq, M., Wagenbrenner, J., Stacy, E., Hartsough, P., Devine, S., Tian, Z., Ferrell, R., Goulden, M., Hopmans, J.W., Bales, R., 2018. Southern sierra critical zone observatory and kings river experimental watersheds: a synthesis of measurements, new insights, and future directions. Vadose Zone J. 17 (1), 1–18. <https://doi.org/10.2136/vzj2018.04.0081>.
- Pizzo, M., Handl, R., Zurmuehl, M. (Eds.), 2020. Odata Version 4.01. Part 1: Protocol, OASIS Standard. <https://docs.oasis-open.org/odata/odata/v4.01/os/part1-protocol/odata-v4.01-os-part1-protocol.html>. <https://docs.oasis-open.org/odata/odata/v4.01/odata-v4.01-part1-protocol.html>.
- Sadler, J.M., Ames, D.P., Khattar, R., 2016. A recipe for standards-based data sharing using open source software and low-cost electronics. Hydroinformatics 18 (2), 185–197. <https://doi.org/10.2166/hydro.2015.092>.
- Scarbrough, K., Persaud, P., Fletcher, I., Akin, A.A., Hathaway, J., Khojandi, A., 2023. Real-time sensor-based prediction of soil moisture in green infrastructure: a case study. Environ. Model. Software 162, 105638. <https://doi.org/10.1016/j.envsoft.2023.105638>.
- Schmidt, L., Schäfer, D., Geller, J., Lünenschloss, P., Palm, B., Rinke, K., Rebmann, C., Rode, M., Bumberger, J., 2023. System for automated quality control (SaQC) to enable traceable and reproducible data streams in environmental science. Environ. Model. Software 169, 105809. <https://doi.org/10.1016/j.envsoft.2023.105809>.
- Simonis, I., Echterhoff, J., 2011. OGC sensor planning service implementation standard. Open Geospatial Consortium. Implement. Standard. OGC 09-000, Version 2.0. <http://www.opengis.net/doc/IS/SPS/2.0>.
- Spackman Jones, A., Horsburgh, J.S., 2025. Hydrologic information systems: An introductory overview. Environ. Model. Software 185, 106308. <https://doi.org/10.1016/j.envsoft.2024.106308>.
- Tarboton, D.G., Ames, D.P., Horsburgh, J.S., Goodall, J.L., Couch, A., Hooper, R., Bales, J., Wang, S., Castronova, A., Seul, M., Idaszak, R., Li, Z., Dash, P., Black, S., Ramirez, M., Yi, H., Calloway, C., Cogswell, C., 2024. HydroShare retrospective: a review of science and technology advances of a comprehensive data and model publication environment for the water science domain. Environ. Model. Software 172, 105902. <https://doi.org/10.1016/j.envsoft.2023.105902>.
- Tarboton, D.G., Horsburgh, J.S., Maidment, D.R., Whiteaker, T., Zaslavsky, I., Piasecki, M., Goodall, J., Valentine, D., Whitenack, T., 2009. Development of a community hydrologic information system. In: Anderssen, R.S., Braddock, R.D., Newham, L.T.H. (Eds.), 18th World IMACS Congress and MODSIM09 International Congress on Modelling and Simulation, Modelling and Simulation Society of Australia and New Zealand and International Association for Mathematics and Computers in Simulation, pp. 988–994. [https://mssanz.org.au/modsim09/C4/tarboton\\_C4.pdf](https://mssanz.org.au/modsim09/C4/tarboton_C4.pdf).
- Taylor, P. (Ed.), 2014. OGC Waterml 2.0: Part 1 - Timeseries, Open Geospatial Consortium Implementation Standard, pp. 10–126r4. OGC, Version 2.0.1. <http://www.opengis.net/doc/IS/waterml/2.0.1>.
- Taylor, P. (Ed.), 2015. OGC Waterml 2.0: Part 2 – Ratings, Gaugings and Sections, Open Geospatial Consortium, pp. 15–18. OGC, Version 1.0. <https://docs.ogc.org/is/15-018r2/15-018r2.html>.
- Trevathan, J., Schmidtke, S., 2022. Open-source Internet of Things remote aquatic environmental sensing. HardwareX 12, e00336. <https://doi.org/10.1016/j.ohx.2022.e00336>.
- Wong, B.P., Kerkez, B., 2016. Real-time environmental sensor data: an application to water quality using web services. Environ. Model. Software 84, 505–517. <https://doi.org/10.1016/j.envsoft.2016.07.020>.
- Zaslavsky, I., Valentine, D., Whiteaker, T. (Eds.), 2007. CUAHSI Waterml, Open Geospatial Consortium Discussion Paper. Version 0.3.0, OGC 07-041r1. [https://portal.ogc.org/files/?artifact\\_id=21743](https://portal.ogc.org/files/?artifact_id=21743).